Department of Social Welfare and Development
Pantawid Pamilyang Pilipino Program (4Ps), Conditional Cash Transfer Group

OpenSPP and Odoo Foundations Training

Participant Guide

Modules 3 to 6: Introduction to Odoo, Developing Custom Modules and Extending Odoo Models, DSWD-specific implementation of OpenSPP, and the Hands-on Activity

Edwin Gonzales
Head of Engineering, OpenSPP

Edition of 9 November 2026, from commit e533f7f

Contents

About this guide **4**

0. Before you start **5**

0.1 What you have 5
0.2 The map of your repository 5
0.3 The nine commands 5
0.4 The edit loop 6

Module 3: Introduction to Odoo **7**

3.1 What you can do after this module 7
3.2 What Odoo is 7
3.3 MVC, in Odoo terms 7
3.4 Odoo in five words 8
3.5 The module you build: `dswd_asset` 8
3.6 Step 0: a module is a folder and a manifest 9
3.7 Step 1: the model is the table 10
3.8 Fields in depth 11
3.9 Step 2: views draw the table, a menu opens it 15
3.10 Wizards and actions returned from Python 16
3.11 Step 3: groups and access rows 17
3.12 Record rules: who sees which rows 18
3.13 Step 4: tests say what "working" means 18
3.14 Step 5: the ORM, from a method and from the shell 20
3.15 The ORM in module code 22
3.16 Model methods: overrides, errors, logs and translations 24
3.17 Scheduled actions and background jobs 25
3.18 Step 6: a PDF report 26
3.19 The four mistakes of a new module 27
3.20 From Module 3 to Module 4 27

Module 4: Developing Custom Modules and Extending Odoo Models **28**

4.1 What you can do after this module 28
4.2 The three rules 28
4.3 The requirement 28
4.4 Vocabularies: a code list as data 28
4.5 The field on the household 30
4.6 Three kinds of inheritance 30
4.7 The view: one XPath on a named anchor 31
4.8 View inheritance: every position 31
4.9 The security file 32
4.10 The tests 32
4.11 The five mistakes of an extension 33
4.12 Where this lives in production 33
4.13 Upgrading a module that is already in production 33
4.14 Exercises (Module 6, Day 3) 34

Module 5: DSWD-specific implementation of OpenSPP 35
5.1 What you can do after this module 35
5.2 How to read a production module 35
5.3 The map 36
5.4 Foundation: where, who, who may, when 36
5.5 Change requests: the Beneficiary Update System 37
5.6 Compliance and Family Development Sessions 38
5.7 Payroll: what not to break 38
5.8 Casework and the bridge 38
5.9 Integration 38
5.10 Reading production code 39
5.11 How to approach a 4Ps ticket 39
5.12 Not today 40
5.13 Claude Code in development 40

Module 6: Hands-on Activity 42
6.1 What you can do after this module 42
6.2 Before 8:30 42
6.3 The three exercises 42
6.4 Done means three proofs 43
6.5 When something goes wrong 43
6.6 After the training 44

7. Reference 45
7.1 Naming rules of this repository 45
7.2 Odoo 19 notes 45
7.3 Things that break the whole stack 45
7.4 From a change to a merge 46
7.5 Where to read more 47
7.6 Glossary 47

About this guide

DSWD Pantawid Pamilyang Pilipino Program (4Ps), Conditional Cash Transfer Group. This guide is the written form of the training: read it before a session to know what is coming, follow it during the hands-on parts, and keep it as the reference for the module development and support work that follows the training. It contains every command, every file and every error message the sessions use. It contains no screenshots: everything in it can be reproduced on your own laptop with your own repository.

Modules 1 and 2 are added as their sessions are prepared. This edition covers Modules 3 to 6: on Day 2, Module 3, Introduction to Odoo, and Module 4, Developing Custom Modules and Extending Odoo Models, in the morning, and Module 5, DSWD-specific implementation of OpenSPP, in the afternoon; on Day 3, Module 6, the hands-on activity.

0. Before you start

0.1 What you have

If you followed `training/SETUP.md`, you have:

- Docker Desktop, Git, Python, pre-commit and Claude Code installed and verified;
- your own repository, created from the template with "Use this template", cloned to your laptop;
- the OpenSPP image pulled, the OpenSPP source in `upstream/openspp/`, and a first start that ended at a styled login page.

Everything in this guide happens inside that repository folder. On Windows PowerShell, type `.\odoo-project` (or `./odoo-project`, which also works) wherever this guide writes `./odoo-project`; a bare `odoo-project` does not work. Paths are written with `/`; PowerShell accepts them too.

The few `grep` commands in this guide search files. Windows PowerShell has no `grep`; use `Select-String -Path <file> -Pattern <text>` instead, or search the folder in VS Code (Ctrl+Shift+F). `git grep` does not see `upstream/openspp/`, because that folder is not tracked by Git. Bash habits such as `&&`, `||` and `> /dev/null` do not work in Windows PowerShell 5.1 either; run one command per line.

0.2 The map of your repository

Where	What
`dswd_*/` at the root	Your modules. Every folder with `__manifest__.py` at the root is an Odoo module.
`dswd_asset/`	The Module 3 module: an IT asset register built from scratch.
`dswd_wellbeing/`	The Module 4 module: a 4Ps well-being level added to households.
`upstream/openspp/`	The OpenSPP source for the release you run, read-only. Never edit it; read it.
`training/`	The setup guide, this guide, and the per-module materials.
`docs/`	Guides, principles, architecture decisions.
`odoo-project`	The command-line tool that drives Docker for you.

The repository root is mounted into the container as an addons folder. That is why your modules sit at the root, and why no folder at the root may be called `forms`, `helpers`, `mixins`, `utilities` or `vendor` (section 7.3).

0.3 The nine commands

When	Run
Start the stack	`./odoo-project start`
Something is wrong before you even begin	`./odoo-project doctor`
Install a new module into the running database	`./odoo-project install <module>`
A module you already installed changed	`./odoo-project update`
You changed Python and run the default profile	`./odoo-project restart`
Read what the running server says	`./odoo-project logs`
Run a module's tests on a clean database	`./odoo-project test <module>`
Pages lost their styling	`./odoo-project resetassets`
End of the day	`./odoo-project stop`

`install`, `update` and `test` print their output, errors included, in the terminal where you ran them; `logs` shows only the running server, not that output.

Login: `admin` / `admin`. Developer mode: Settings, scroll to the bottom, "Activate the developer mode". Developer mode adds the Technical menu, which is the x-ray of everything in this guide.

0.4 The edit loop

Odoo reads your files when it installs or updates a module, not continuously. After a change:

You changed	Run
Nothing yet: a brand-new module	`./odoo-project install <module>`
`__manifest__.py`, a data file, a security file, a new field, a new view record	`./odoo-project update`
A Python method body	`./odoo-project restart` (nothing with `start --profile dev`)
An existing view's XML	`./odoo-project update` (nothing with `start --profile dev`)
A record in a `noupdate="1"` data file	a fresh database (`stop -v -y`, `start`, `install`); 4.4
A test	`./odoo-project test <module>`, then read the summary line

`update` never installs a new module; `install` does. Both restart the server.

`start --profile dev` reloads Python and view changes by itself. It runs on its own port, not 8069: it prints its URL, and `./odoo-project url` shows it again. On Windows, with the repository on the `C:` drive, the dev profile may not see your edits at all (the container gets no file-change events from the Windows drive); if a change does not show, use `restart` and `update` as on the default profile.

The edit loop is the inner part of a longer path that every change takes: a branch, the loop above, a commit with the pre-commit hooks, a push, a pull request, CI on GitHub, a review and a merge. Section 7.4, "From a change to a merge", gives the whole path step by step; Module 4 shows it, and Module 6 has you do it.

Module 3: Introduction to Odoo

The session builds `dswd_asset` in seven steps, numbered 0 to 6. Your repository holds the finished module, with every step in it. Between the steps, sections marked **Reference** go deeper into the topic the step introduced: every field type, the ORM as written in module code, wizards, record rules, overrides, scheduled jobs. They are not presented in the 90 minutes; they are what you come back to when you maintain and build modules. Their examples continue the same two subjects, the asset register and 4Ps households, and were run against Odoo 19 and OpenSPP 2026.09 before they were printed here.

3.1 What you can do after this module

1. Say what Odoo is (the platform, its licence, its release, where OpenSPP sits on it) and name the three parts of MVC in Odoo terms.
2. Say what a module is, name the role of each file in one, and say why a file not listed in the manifest does not exist.
3. Write a model with Char, Selection, Date, Many2one and Boolean fields, and say "model = table, field = column".
4. Write a list view, a form view, an action and a menu, and explain the chain menu, action, view, model.
5. Write the access row a new model needs, and recognise its absence in the log and on screen.
6. Use the ORM from a method and from the shell: `search`, `browse`, `create`, `write`, `unlink`.
7. Add a PDF report to a model and find it in the Print menu.
8. Run a module's tests and read the summary line.

3.2 What Odoo is

Odoo is an open-source business application platform written in Python on PostgreSQL, with a web client written in JavaScript. It began in 2005 as TinyERP, became OpenERP, and has been Odoo since 2014. One major version ships every year in the autumn; this training runs on **Odoo 19** (2025). Two editions exist: **Community**, under the LGPL v3 licence, which is what OpenSPP builds on and what you run; and **Enterprise**, proprietary modules sold by Odoo S.A. on top of Community. Everything in this training is Community.

What makes Odoo a platform rather than a product is the module system: every feature, including the core ones, is a module, and a module can add tables, columns, screens, menus, reports and rules to any other module without changing its code. OpenSPP is about a hundred such modules (`spp_*`) that turn Odoo into a social protection management information system: registry, programs, cycles, entitlements, change requests, grievances, an API. Your `dswd_*` modules sit on top of those, the same way OpenSPP sits on top of Odoo.

Three facts to keep:

- The database is PostgreSQL; every model is a table you could inspect with `./odoo-project sql`.
- The server is one Python process (several, in production) that loads every installed module at start.
- The web client is a single-page application; the XML views you write are turned into screens by it.

3.3 MVC, in Odoo terms

Odoo is a Model-View-Controller platform, with its own names for the three parts:

MVC part	In Odoo	Where you write it
Model	`models.Model` classes and their fields; the ORM maps them to tables and rows	`models/*.py`

MVC part	In Odoo	Where you write it
View	XML records on `ir.ui.view` (list, form, search, kanban) and QWeb templates for reports; rendered by the web client	`views/*.xml`, `report/*.xml`
Controller	What happens when the user acts: window actions and menus route clicks to views; methods on the model run the business logic; HTTP controllers serve URLs	`views/*.xml` (actions, menus), `models/*.py` (methods), `controllers/*.py`

The rule of thumb: data and rules live in the model, drawing lives in the view, and the controller is thin. A button on a form calls a method on the model; the method uses the ORM; the view only says where the button sits.

3.4 Odoo in five words

Word	Means	See it at
Module	a folder with `__manifest__.py` on the addons path	Apps
Model	a table: `_name = "dswd.asset"` becomes `dswd_asset`	Settings > Technical > Models
Field	a column with a type	Settings > Technical > Fields
View	how the table is drawn: list, form, search	Settings > Technical > Views
Menu	menu, then action, then view, then model	the top bar and the app tiles

Open Settings > Technical > Models and search `res.partner`. Its fields come from `base`, from `spp_registry`, from `spp_vocabulary` and more. Many modules add columns to one table. Module 4 is about doing exactly that.

3.5 The module you build: `dswd_asset`

An IT asset register: the laptops, desktops and printers of an office and who looks after them. It is plain Odoo; nothing in it is specific to 4Ps. The `dswd_` prefix is this project's naming rule, enforced by the repository's lint, the way `spp_` is OpenSPP's: it says who owns the code, not what it is about.

```
dswd_asset/
├── __manifest__.py                declares the module and lists its files
├── __init__.py                    imports the models package
├── models/__init__.py             imports the model file
├── models/dswd_asset.py           the model: table, fields, a computed field, a constraint
├── views/dswd_asset_views.xml     list, form and search views, and the action
├── views/menus.xml                the app tile and the menu that opens the action
├── report/dswd_asset_report.xml   the Asset Sheet PDF report and its template
├── security/groups.xml            viewer, officer and manager groups
├── security/ir.model.access.csv   who may read, write, create, delete
├── tests/__init__.py               imports the test files
├── tests/test_dswd_asset.py       what "working" means, as code
├── tests/test_access.py           the access rows, proven as each tier
├── tests/test_report.py           the report renders
├── static/description/icon.png    the app tile icon
├── pyproject.toml                 packaging metadata
└── readme/DESCRIPTION.md          what the module is for
```

The module ships in your repository with `auto_install: False`. It is installed only when you run `./odoo-project install dswd_asset`.

3.6 Step 0: a module is a folder and a manifest

```
{
    "name": "DSWD Training: IT Asset Register",
    "summary": "A plain Odoo module built from scratch in Module 3: an IT asset register",
    "category": "DSWD 4Ps/Training",
    "version": "19.0.1.0.0",
    "author": "DSWD",
    "website": "https://github.com/pithtech/OpenSPP-Template",
    "license": "LGPL-3",
    "development_status": "Alpha",
    "maintainers": [],
    "depends": [
        "base",
        "spp_security",
    ],
    "data": [
        "security/ir.model.access.csv",
    ],
    "application": False,
    "installable": True,
    "auto_install": False,
}
```

This is the manifest at step 0. At this point `security/ir.model.access.csv` holds only its header line, and `data` lists nothing else, because nothing else exists yet. Each later step adds its file to `data`; the finished manifest in your repository lists five:

```
    "data": [
        "security/groups.xml",
        "security/ir.model.access.csv",
        "views/dswd_asset_views.xml",
        "views/menus.xml",
        "report/dswd_asset_report.xml",
    ],
```

- `depends` lists the modules Odoo must install first. `base` is Odoo itself; `spp_security` is the OpenSPP module that owns the admin group the manager tier links into (step 3).
- `data` lists the files Odoo loads, in order. **A file not listed here does not exist to Odoo.** No error is raised; the file is simply ignored. The reverse fails: a file listed in `data` that does not exist stops the install.
- `installable: True` makes it appear in Apps; `auto_install: False` keeps it out of the automatic first start. `application: False` means the default "Apps" filter hides it.

With this manifest, the header-only access file and an empty `__init__.py`, the module installs. It does nothing yet. From the terminal: `./odoo-project install dswd_asset`; its output, in the same terminal, shows `Loading module dswd_asset`. Or in Apps: turn on developer mode, click Update Apps List, remove the "Apps" filter from the search box (the x on the facet), search "Asset", and install; then `./odoo-project logs` shows the same line.

3.7 Step 1: the model is the table

```
from odoo import _, api, fields, models
from odoo.exceptions import ValidationError


class DswdAsset(models.Model):
    _name = "dswd.asset"
    _description = "IT Asset"
    _order = "name"

    name = fields.Char(string="Asset Tag", required=True)
    model_name = fields.Char(string="Make and Model")
    asset_type = fields.Selection(
        [("laptop", "Laptop"), ("desktop", "Desktop"), ("printer", "Printer"), ("other", "Other")],
        required=True,
        default="laptop",
    )
    custodian_id = fields.Many2one("res.users", string="Custodian", default=lambda self: self.env.user)
    purchase_date = fields.Date()
    warranty_end_date = fields.Date()
    is_in_use = fields.Boolean(default=True)
    is_under_warranty = fields.Boolean(compute="_compute_is_under_warranty")

    @api.depends("warranty_end_date")
    def _compute_is_under_warranty(self):
        today = fields.Date.context_today(self)
        for asset in self:
            asset.is_under_warranty = bool(asset.warranty_end_date and asset.warranty_end_date >= today)

    @api.constrains("purchase_date", "warranty_end_date")
    def _check_warranty_after_purchase(self):
        for asset in self:
            if asset.purchase_date and asset.warranty_end_date and asset.warranty_end_date < asset.purchase_date:
                raise ValidationError(_("The warranty end date must be on or after the purchase date."))
```

The file in your repository also carries a class docstring and comments, left out here. What each line teaches:

Line	Teaches
`_name = "dswd.asset"`	A new table, `dswd_asset`. `_description` is required by this repository; Odoo warns without it.
`fields.Char(required=True)`	A text column; `required` is the first validation a user meets.
`fields.Selection([...])`	A fixed list of values **in code**. Adding one is a code change and a release (section 4.4).
`fields.Many2one("res.users")`	A link to another table. The dropdown and the avatar come for free.
`fields.Date()`	A real date with a picker, not text.
`fields.Boolean(default=True)`	A checkbox; `default` applies to new records.
`compute=` with `@api.depends`	A value derived from other fields, recalculated when they change, never typed in. Not stored, so it cannot be searched or filtered unless it has a `search=` method (3.8).
`@api.constrains`	A rule that refuses bad data with a sentence the user reads.

Every field type, related and computed fields in depth, defaults, attributes and constraints: section 3.8.

The model needs two import lines to be seen: `from . import dswd_asset` in `models/__init__.py` and `from . import models` in `__init__.py`. Forgetting either raises no error; the model is simply absent. After `./odoo-project update`, Settings > Technical > Fields, searched by model `dswd.asset`, lists the columns: fourteen of them, the eight you wrote and six Odoo adds to every table (`id`, `display_name`, who and when created and last updated).

The output of that `update`, in the same terminal, also says:

```
WARNING ... odoo.modules.loading: The models ['dswd.asset'] have no access rules in module dswd_asset, consider adding some, like:
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
dswd_asset.access_dswd_asset,access_dswd_asset,dswd_asset.model_dswd_asset,base.group_user,1,0,0,0
```

Odoo even suggests a row. Nothing is broken. Nobody can open the table either.

3.8 Fields in depth

Reference: not presented in the session. Read it after step 1.

Simple types

```
class DswdAsset(models.Model):
    _name = "dswd.asset"
    _inherit = ["dswd.asset", "mail.thread"]          # mail.thread is needed for tracking=

    serial_number = fields.Char(copy=False, tracking=True, index="trigram")
    notes = fields.Text()
    description = fields.Html(sanitize=True)
    quantity = fields.Integer(default=1)
    weight_kg = fields.Float(digits=(10, 2))
    currency_id = fields.Many2one("res.currency", required=True, default=lambda self: self.env.company.currency_id)
    purchase_cost = fields.Monetary(currency_field="currency_id", aggregator="sum")
    last_seen_at = fields.Datetime(readonly=True)
    condition = fields.Selection([("good", "Good"), ("repair", "Needs repair")], default="good", required=True)
    receipt = fields.Binary(attachment=True)
    receipt_filename = fields.Char()
    photo = fields.Image(max_width=1024, max_height=1024)
```

Type	Stores	Note
`Char`	short text	`size=` is rarely needed
`Text`	long text	shown as a text area
`Html`	formatted text	`sanitize=True` strips scripts from user input
`Integer`	whole numbers	
`Float`	decimals	`digits=(total, decimals)`; never use Float for money
`Monetary`	money	needs a currency field, named in `currency_field=`
`Boolean`	true or false	name it `is_`, `has_` or `can_`
`Date`	a date	`fields.Date.context_today(self)` for "today" in the user's timezone
`Datetime`	a moment	stored in UTC, shown in the user's timezone
`Selection`	one of a fixed list	for internal states only; code lists use vocabularies (4.4)
`Binary`	a file	`attachment=True` keeps it out of the table; add a filename field
`Image`	a picture	resized on save to `max_width` / `max_height`

`aggregator="sum"` makes list views total the column when grouped (it replaced `group_operator`). A default given as `default=fields.Date.today()` is computed once, when the server starts; pass the function itself or a lambda.

Relational types

```
custodian_id = fields.Many2one("res.users", ondelete="set null", index=True)
household_id = fields.Many2one(
    "res.partner",
    ondelete="restrict",
    domain=[("is_registrant", "=", True), ("is_group", "=", True)],
)
movement_ids = fields.One2many("dswd.asset.movement", "asset_id")        # the other side of a Many2one
tag_ids = fields.Many2many("spp.vocabulary.code", "dswd_asset_tag_rel", "asset_id", "code_id")
```

```
class DswdAssetMovement(models.Model):
    _name = "dswd.asset.movement"
    _description = "Asset Movement"

    asset_id = fields.Many2one("dswd.asset", required=True, ondelete="cascade", index=True)
    moved_on = fields.Date(default=fields.Date.context_today)
    to_location = fields.Char(required=True)
```

- **Many2one**: a column holding the id of another record. `ondelete` decides what happens when that record is deleted: `set null` (the default; `restrict` when the field is required), `restrict` (refuse the deletion), `cascade` (delete this record too). Index Many2one fields you search or group on.
- **One2many**: no column; the list of records whose Many2one points back here. It needs that Many2one to exist on the other model.
- **Many2many**: a link table. Name the table and its two columns explicitly when two Many2many fields could link the same two models.
- A `domain` on a relational field only filters the dropdown (4.5).

Related fields

A related field shows a field of a linked record as if it were on this one.

```
custodian_email = fields.Char(related="custodian_id.email")          # read through the link, no column
household_level_code = fields.Char(
    related="household_id.dswd_well_being_level_id.code",
    store=True,                                                       # a real column, kept up to date
    index=True,
)
```

- Not stored: computed on read, cannot be used in a search or group by of large tables efficiently, costs nothing to add.
- Stored: a column Odoo updates whenever the source changes; searchable, sortable, groupable. Use it for list views and filters.
- A related field is read-only unless you set `readonly=False`; then writing it writes the **linked** record.

Computed fields

```
# Not stored: recalculated whenever it is read
is_under_warranty = fields.Boolean(compute="_compute_is_under_warranty")

@api.depends("warranty_end_date")
def _compute_is_under_warranty(self):
    today = fields.Date.context_today(self)
    for asset in self:                          # always loop, always assign on every record
        asset.is_under_warranty = bool(asset.warranty_end_date and asset.warranty_end_date >= today)
```

Stored, with a dotted dependency. The value is a column; Odoo recomputes it when anything listed in `depends` changes, including on linked records.

```
class ResPartner(models.Model):
    _inherit = "res.partner"

    dswd_member_count = fields.Integer(compute="_compute_dswd_member_count", store=True)

    @api.depends("group_membership_ids.is_ended")
    def _compute_dswd_member_count(self):
        for partner in self:
            partner.dswd_member_count = len(partner.group_membership_ids.filtered(lambda m: not m.is_ended))
```

- Forget a dependency and the value goes stale silently: here, without `.is_ended`, ending a membership would not update the count.
- A stored field added to `res.partner` is computed for **every** partner on the `update` that installs it. With millions of registrants that is an upgrade window, not a quick update.
- A compute must assign a value on every record, even when there is nothing to compute.

Editable: `inverse`. The user types the computed value; the inverse writes it back to the real fields.

```
from dateutil.relativedelta import relativedelta

warranty_months = fields.Integer(compute="_compute_warranty_months", inverse="_inverse_warranty_months")

@api.depends("purchase_date", "warranty_end_date")
def _compute_warranty_months(self):
    for asset in self:
        if asset.purchase_date and asset.warranty_end_date:
            delta = relativedelta(asset.warranty_end_date, asset.purchase_date)
            asset.warranty_months = delta.years * 12 + delta.months
        else:
            asset.warranty_months = 0

def _inverse_warranty_months(self):
    for asset in self.filtered("purchase_date"):
        asset.warranty_end_date = asset.purchase_date + relativedelta(months=asset.warranty_months)
```

Searchable without storing: `search`. A non-stored computed field cannot be filtered (3.7), unless it has a search method that translates the condition into a domain on stored fields.

```
is_under_warranty = fields.Boolean(compute="_compute_is_under_warranty", search="_search_is_under_warranty")

def _search_is_under_warranty(self, operator, value):
    today = fields.Date.context_today(self)
    if operator not in ("in", "not in"):
        return NotImplemented
    if (operator == "in") == (True in value):
        return [("warranty_end_date", ">=", today)]
    return ["|", ("warranty_end_date", "=", False), ("warranty_end_date", "<", today)]
```

In Odoo 19, conditions such as `("is_under_warranty", "=", True)` reach the search method as `("is_under_warranty", "in", [True])`; handle `in` and `not in` and return `NotImplemented` for anything else.

Recursive. A value that depends on the same field of child records.

```
parent_id = fields.Many2one("dswd.asset", string="Installed In", index=True)
child_ids = fields.One2many("dswd.asset", "parent_id")
total_cost = fields.Monetary(compute="_compute_total_cost", store=True, recursive=True, currency_field="currency_id")

@api.depends("purchase_cost", "child_ids.total_cost")
def _compute_total_cost(self):
    for asset in self:
        asset.total_cost = asset.purchase_cost + sum(asset.child_ids.mapped("total_cost"))
```

Defaults

```
custodian_id = fields.Many2one("res.users", default=lambda self: self.env.user)
quantity = fields.Integer(default=1)
```

From an action or from code, `context={"default_asset_type": "printer"}` sets a default for one form or one create. A wizard that starts from selected records fills itself in `default_get` (3.10).

Field attributes

Attribute	Effect
`required=True`	NOT NULL in the database; the form refuses to save without it
`readonly=True`	not editable in forms; code can still write it
`index=True`	a database index; also `"btree_not_null"`, and `"trigram"` for `ilike`
`copy=False`	not copied by Duplicate (serial numbers, references, states)
`tracking=True`	changes logged in the chatter; the model must inherit `mail.thread`
`groups="dswd_asset.group_asset_manager"`	the field exists only for members of the group: field-level security
`string=`, `help=`	the label, and the tooltip shown with the `?`

Constraints: SQL and Python

```
# Enforced by PostgreSQL (Odoo 19 syntax)
_unique_serial = models.Constraint("UNIQUE(serial_number)", "The serial number must be unique.")
_positive_cost = models.Constraint("CHECK(purchase_cost >= 0)", "The purchase cost cannot be negative.")
_custodian_type_idx = models.Index("(custodian_id, asset_type)")

# Enforced by Odoo when a listed field is written
@api.constrains("quantity")
def _check_quantity(self):
    for asset in self:
        if asset.quantity < 1:
            raise ValidationError(_("The quantity must be at least 1."))
```

- Use SQL constraints for uniqueness and simple checks on one row. If existing rows break a new constraint, Odoo logs it and does not create it; clean the data first.
- Use `@api.constrains` for rules across fields or records. List plain field names only (no dotted paths); it runs only when one of them is written.
- `_sql_constraints`, the pre-19 list of tuples, is no longer supported and only logs a warning.

Onchange

```
@api.onchange("asset_type")
def _onchange_asset_type(self):
    """Suggest a one-year warranty for printers. Runs in the form only."""
    if self.asset_type == "printer" and self.purchase_date and not self.warranty_end_date:
        self.warranty_end_date = fields.Date.add(self.purchase_date, years=1)
    if self.asset_type == "other":
        return {"warning": {"title": _("Check the type"), "message": _("A specific type helps reporting.")}}
```

An onchange runs only while a user edits a form. Imports, the API, change requests and code never run it; a value that must always be right belongs in a computed field (with `store=True, readonly=False` if users may override it), a constraint, or a `create`/`write` override.

3.9 Step 2: views draw the table, a menu opens it

```
<record id="view_dswd_asset_list" model="ir.ui.view">
    <field name="name">dswd.asset.list</field>
    <field name="model">dswd.asset</field>
    <field name="arch" type="xml">
        <list>
            <field name="name" />
            <field name="model_name" />
            <field name="asset_type" />
            <field name="custodian_id" widget="many2one_avatar_user" />
            <field name="purchase_date" optional="show" />
            <field name="is_under_warranty" />
        </list>
    </field>
</record>

<record id="view_dswd_asset_form" model="ir.ui.view">
    <field name="name">dswd.asset.form</field>
    <field name="model">dswd.asset</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <div class="oe_title">
                    <h1><field name="name" placeholder="Asset tag, e.g. LT-0001" /></h1>
                </div>
                <group name="group_main">
                    <group name="group_identity" string="Identity">
                        <field name="model_name" />
                        <field name="asset_type" />
                        <field name="custodian_id" widget="many2one_avatar_user" />
                    </group>
                    <group name="group_lifecycle" string="Lifecycle">
                        <field name="purchase_date" />
                        <field name="warranty_end_date" />
                        <field name="is_under_warranty" />
                        <field name="is_in_use" />
                    </group>
                </group>
            </sheet>
        </form>
    </field>
</record>

<record id="view_dswd_asset_search" model="ir.ui.view">
    <field name="name">dswd.asset.search</field>
    <field name="model">dswd.asset</field>
    <field name="arch" type="xml">
        <search>
            <field name="name" />
            <field name="model_name" />
            <field name="custodian_id" />
            <filter name="filter_in_use" string="In use" domain="[('is_in_use', '=', True)]" />
            <group>
                <filter name="group_by_type" string="Type" context="{'group_by': 'asset_type'}" />
                <filter name="group_by_custodian" string="Custodian" context="{'group_by': 'custodian_id'}" />
            </group>
        </search>
    </field>
</record>

<record id="action_dswd_asset" model="ir.actions.act_window">
    <field name="name">IT Assets</field>
    <field name="res_model">dswd.asset</field>
    <field name="view_mode">list,form</field>
    <field name="context">{'search_default_filter_in_use': 1}</field>
    <!-- a help field, the text shown on an empty list, follows; see the file -->
</record>
```

Abridged: comments and the action's `help` text are left out; the full files are `dswd_asset/views/dswd_asset_views.xml` and `dswd_asset/views/menus.xml`.

```
<menuitem id="menu_dswd_asset_root" name="IT Assets" web_icon="dswd_asset,static/description/icon.png" sequence="95" />
<menuitem id="menu_dswd_asset" name="Assets" parent="menu_dswd_asset_root" action="action_dswd_asset" sequence="10" />
```

- A view is a record of the model `ir.ui.view` whose `arch` is the XML that draws it. Three kinds here: `list` (rows), `form` (one record), `search` (the search box, filters and group-bys).
- In Odoo 19 the list view's tag is `<list>`; older code wrote `<tree>`.
- Named `<group>` elements in the form (`group_identity`, `group_lifecycle`) are the anchors another module could extend. Module 4 relies on upstream views having such names.
- An action says which model and which views; a menu points at the action. A top-level menu with a `web_icon` is an app tile.
- XML ids follow the repository's rules: `view_<model>_<type>`, `action_<model>`, `menu_<model>_<purpose>`.

After `./odoo-project update`, nothing new appears, not even for `admin`: Odoo hides a menu whose model you may not read, and `admin` is not the superuser. Open the action directly instead, at `http://localhost:8069/odoo/action-dswd_asset.action_dswd_asset` (with the dev profile, use the port that `start` printed). You get:

```
Access Error
You are not allowed to access 'IT Asset' (dswd.asset) records.

No group currently allows this operation.

Contact your administrator to request access if necessary.
```

The warning from step 1 became an error the moment there was a door to knock on. The tile appears at step 3, once the access rows exist.

3.10 Wizards and actions returned from Python

Reference: not presented in the session. Read it after step 2.

A wizard is a temporary model (`TransientModel`) behind a dialog; a method can return an action that tells the web client what to open next.

```
from odoo import Command, _, api, fields, models


class DswdAssetReassignWizard(models.TransientModel):
    _name = "dswd.asset.reassign.wizard"
    _description = "Reassign Assets"

    asset_ids = fields.Many2many("dswd.asset")
    custodian_id = fields.Many2one("res.users", required=True)

    @api.model
    def default_get(self, fields_list):
        values = super().default_get(fields_list)
        if "asset_ids" in fields_list and self.env.context.get("active_model") == "dswd.asset":
            values["asset_ids"] = [Command.set(self.env.context.get("active_ids", []))]   # the rows ticked in the list
        return values

    def action_apply(self):
        self.asset_ids.write({"custodian_id": self.custodian_id.id})
        return {
            "type": "ir.actions.act_window",
            "name": _("Reassigned assets"),
            "res_model": "dswd.asset",
            "view_mode": "list,form",
            "domain": [("id", "in", self.asset_ids.ids)],
        }
```

```
<record id="action_dswd_asset_reassign_wizard" model="ir.actions.act_window">
    <field name="name">Reassign</field>
    <field name="res_model">dswd.asset.reassign.wizard</field>
    <field name="view_mode">form</field>
    <field name="target">new</field>                                    <!-- a dialog -->
    <field name="binding_model_id" ref="dswd_asset.model_dswd_asset" />  <!-- in the Action menu of the list -->
</record>
```

```
access_dswd_asset_reassign_wizard_officer,reassign wizard officer,model_dswd_asset_reassign_wizard,dswd_asset.group_asset_officer,1,1,1,1
```

A transient model needs an access row like any other model. The same dictionary shape opens a form, a filtered list or another wizard from any button:

```
def action_open_movements(self):
    self.ensure_one()
    return {
        "type": "ir.actions.act_window",
        "name": _("Movements of %s", self.name),
        "res_model": "dswd.asset.movement",
        "view_mode": "list,form",
        "domain": [("asset_id", "=", self.id)],
        "context": {"default_asset_id": self.id},
    }
```

3.11 Step 3: groups and access rows

```
<record id="category_dswd_asset" model="ir.module.category">
    <field name="name">DSWD Training: IT Assets</field>
    <field name="sequence">60</field>
</record>

<record id="privilege_asset_officer" model="res.groups.privilege">
    <field name="name">IT Assets</field>
    <field name="category_id" ref="category_dswd_asset" />
</record>

<record id="group_asset_viewer" model="res.groups">
    <field name="name">IT Assets: Viewer</field>
    <field name="privilege_id" ref="privilege_asset_officer" />
    <field name="implied_ids" eval="[Command.link(ref('base.group_user'))]" />
</record>

<record id="group_asset_officer" model="res.groups">
    <field name="name">IT Assets: Officer</field>
    <field name="privilege_id" ref="privilege_asset_officer" />
    <field name="implied_ids" eval="[Command.link(ref('group_asset_viewer'))]" />
</record>

<record id="group_asset_manager" model="res.groups">
    <field name="name">IT Assets: Manager</field>
    <field name="privilege_id" ref="privilege_asset_officer" />
    <field name="implied_ids" eval="[Command.link(ref('group_asset_officer'))]" />
</record>

<record id="spp_security.group_spp_admin" model="res.groups">
    <field name="implied_ids" eval="[Command.link(ref('group_asset_manager'))]" />
</record>
```

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_dswd_asset_system,dswd.asset system,model_dswd_asset,base.group_system,1,1,1,1
access_dswd_asset_viewer,dswd.asset viewer,model_dswd_asset,group_asset_viewer,1,0,0,0
access_dswd_asset_officer,dswd.asset officer,model_dswd_asset,group_asset_officer,1,1,1,0
access_dswd_asset_manager,dswd.asset manager,model_dswd_asset,group_asset_manager,1,1,1,1
```

- Three tiers is this project's convention: a viewer reads; an officer reads, writes and creates; a manager also deletes. `implied_ids` makes each tier include the one below, and the viewer include every internal user.
- A privilege and a category are what Odoo 19 shows on the user form, under Settings > Users, so an administrator can assign a tier with a dropdown.
- The last record adds a line to a group that another module owns, `spp_security.group_spp_admin`: whoever is an OpenSPP admin is an asset manager, without extra setup. (`admin` could open the list even without it, because `admin` is in `base.group_system` and the first access row gives that group every right; the link is what makes `admin` a manager, so it also sees the officer-only button of step 5.)
- The access file: one row per group and model. The `base.group_system` row comes first, always. `model_dswd_asset` is `model_` plus `_name` with dots turned into underscores; a typo there fails at install with `No matching record found for external id`.
- `groups.xml` is listed in the manifest **before** the access file, because the access rows refer to the groups.

After `./odoo-project update`, the list opens. Register the laptop in front of you. Then set a warranty end date earlier than the purchase date and read the dialog: that sentence is in the Python file.

3.12 Record rules: who sees which rows

Reference: not presented in the session. Read it after step 3.

The access file says what a group may do on a model; a record rule says **which rows**.

```
<record id="rule_dswd_asset_officer_own" model="ir.rule">
    <field name="name">IT assets: officers see the assets they hold</field>
    <field name="model_id" ref="dswd_asset.model_dswd_asset" />
    <field name="domain_force">[('custodian_id', '=', user.id)]</field>
    <field name="groups" eval="[Command.link(ref('dswd_asset.group_asset_officer'))]" />
</record>

<record id="rule_dswd_asset_manager_all" model="ir.rule">
    <field name="name">IT assets: managers see all</field>
    <field name="model_id" ref="dswd_asset.model_dswd_asset" />
    <field name="domain_force">[(1, '=', 1)]</field>
    <field name="groups" eval="[Command.link(ref('dswd_asset.group_asset_manager'))]" />
</record>
```

```
def test_officer_sees_only_held_assets(self):
    mine = self.env["dswd.asset"].with_user(self.officer).search([])
    self.assertEqual(mine, self.held_by_officer)
```

- Rules attached to groups are combined with OR: a manager is also an officer, and sees everything because of the second rule. Rules without a group (global rules) are combined with AND and apply to everyone.
- Tests run as the superuser, which ignores record rules, so **test with `with_user`**. The ticket "user X cannot see household Y" is almost always a rule: OpenSPP restricts registrants by area (`spp_area` rules on `area_id child_of` the user's areas).

3.13 Step 4: tests say what "working" means

The tests were written with each step, from step 1 on; step 4 is where the session reads them. Abridged: the full files are `dswd_asset/tests/test_dswd_asset.py` and `dswd_asset/tests/test_access.py`, which hold more tests than shown here.

```
from datetime import date, timedelta

from odoo.exceptions import ValidationError
from odoo.tests import TransactionCase, tagged


@tagged("post_install", "-at_install")
class TestDswdAsset(TransactionCase):
    def setUp(self):
        super().setUp()
        self.Asset = self.env["dswd.asset"]
        self.asset = self.Asset.create({"name": "LT-0001", "model_name": "Dell Latitude 5440"})

    def test_defaults(self):
        self.assertEqual(self.asset.asset_type, "laptop")
        self.assertTrue(self.asset.is_in_use)
        self.assertEqual(self.asset.custodian_id, self.env.user)

    def test_warranty_flag_follows_the_end_date(self):
        self.asset.warranty_end_date = date.today() + timedelta(days=30)
        self.assertTrue(self.asset.is_under_warranty)
        self.asset.warranty_end_date = date.today() - timedelta(days=1)
        self.assertFalse(self.asset.is_under_warranty)

    def test_warranty_before_purchase_is_refused(self):
        with self.assertRaises(ValidationError):
            self.asset.write({"purchase_date": date(2026, 1, 10), "warranty_end_date": date(2025, 12, 31)})
```

```
from odoo import Command
from odoo.exceptions import AccessError
from odoo.tests import TransactionCase, tagged


@tagged("post_install", "-at_install")
class TestDswdAssetAccess(TransactionCase):
    def setUp(self):
        super().setUp()
        self.viewer = self._make_user("asset_viewer", "dswd_asset.group_asset_viewer")
        self.officer = self._make_user("asset_officer", "dswd_asset.group_asset_officer")
        self.manager = self._make_user("asset_manager", "dswd_asset.group_asset_manager")
        self.asset = self.env["dswd.asset"].create({"name": "PR-0001", "asset_type": "printer"})

    def _make_user(self, login, group_xmlid):
        return self.env["res.users"].create(
            {"name": login, "login": login, "email": f"{login}@example.test",
             "group_ids": [Command.set([self.env.ref(group_xmlid).id])]}
        )

    def test_viewer_reads_but_cannot_create(self):
        assets = self.env["dswd.asset"].with_user(self.viewer)
        self.assertEqual(assets.search_count([("name", "=", "PR-0001")]), 1)
        with self.assertRaises(AccessError):
            assets.create({"name": "LT-0002"})

    def test_officer_creates_and_edits_but_cannot_delete(self):
        assets = self.env["dswd.asset"].with_user(self.officer)
        created = assets.create({"name": "LT-0002"})
        created.write({"model_name": "HP ProBook"})
        with self.assertRaises(AccessError):
            created.unlink()

    def test_manager_deletes(self):
        assets = self.env["dswd.asset"].with_user(self.manager)
        assets.create({"name": "LT-0003"}).unlink()
        self.assertEqual(assets.search_count([("name", "=", "LT-0003")]), 0)
```

- `TransactionCase` gives each test a database transaction that is rolled back afterwards; `setUp` runs before each test. `@tagged("post_install", "-at_install")` runs the tests after every module is installed, which is what this repository requires.
- `self.env["dswd.asset"]` is the model; `create`, `write`, `search_count`, `unlink` are ORM operations (3.14 lists the five you use most). `with_user(...)` runs the same operations as that user, which is how the access rows are proven rather than assumed.

```
$ ./odoo-project test dswd_asset
...
========== TEST RESULTS ==========
0 failed, 0 error(s) of 12 tests
Passed: 12 | Failed: 0 | Errors: 0

Full log: .../dswd_asset_unittest_20261108_153000.log
==================================
```

Your repository has all seven steps, so it runs 12 tests; at step 4 alone the count is 8 (steps 5 and 6 add two each). The summary line is the first thing you read when someone says a module is broken; the log path is the second.

3.14 Step 5: the ORM, from a method and from the shell

The ORM (object-relational mapper) is how Python talks to the tables: a model is a class, a row is a record, a query is a method call. Five operations cover almost everything:

Operation	Meaning	Returns
`search(domain)`	the records matching a domain, a list of `(field, operator, value)`	a recordset
`browse(ids)`	the records with these ids, without a query until a field is read	a recordset
`create(values)`	a new row	the new record
`write(values)`	change fields on every record in the set	`True`
`unlink()`	delete every record in the set	`True`

Two methods added to `dswd.asset` show the two shapes a method takes:

```
    def action_assign_to_me(self):
        """Form button: the ORM write in one line. ``self`` is the asset(s) the button was pressed on."""
        self.write({"custodian_id": self.env.user.id})

    @api.model
    def find_warranty_expiring(self, days=30):
        """Assets still in use whose warranty ends within ``days``: a search with a domain.

        ``@api.model`` means the method works on the model, not on records:
        ``env["dswd.asset"].find_warranty_expiring(30)``.
        """
        today = fields.Date.context_today(self)
        return self.search(
            [
                ("is_in_use", "=", True),
                ("warranty_end_date", ">=", today),
                ("warranty_end_date", "<=", fields.Date.add(today, days=days)),
            ],
            order="warranty_end_date",
        )
```

Step 5 also adds a header with the button to the form view:

```
<header>
    <!-- type="object" calls the method of that name on the record -->
    <button
        name="action_assign_to_me"
        type="object"
        string="Assign to me"
        class="btn-primary"
        groups="dswd_asset.group_asset_officer"
    />
</header>
```

- A method without a decorator runs on records: `self` is the asset (or assets) it was called on. The form's button, `type="object"`, calls it on the record being edited. `groups` shows the button only to officers and above: a viewer may not write, so the button would only give them an `AccessError`.
- `@api.model` marks a method that works on the model as a whole: `env["dswd.asset"].find_warranty_expiring(30)`.
- `self.env` is the environment: the current user (`self.env.user`), the other models (`self.env["res.users"]`), xml ids (`self.env.ref("dswd_asset.group_asset_manager")`).

The same operations from the shell, the fastest way to answer "what is in the database right now":

```
$ ./odoo-project shell
>>> Asset = env["dswd.asset"]
>>> Asset.search_count([])
2
>>> laptops = Asset.search([("asset_type", "=", "laptop")])
>>> laptops.mapped("name")
['LT-0001']
>>> a = Asset.create({"name": "LT-0099", "asset_type": "laptop"})
>>> a.write({"model_name": "Lenovo ThinkPad T14"})
True
>>> a.custodian_id.name
'OdooBot'
>>> Asset.find_warranty_expiring(365).mapped("name")
['LT-0001']
>>> a.unlink()
True
>>> env.cr.commit()      # the shell does not save until you say so: shell only, never in module code
```

`mapped`, `filtered` and `sorted` work on any recordset; a Many2one field is itself a record, so `a.custodian_id.name` walks the link. Nothing the shell does is saved until `env.cr.commit()`. Every operation as it is written inside a module (domains, `Command`, `_read_group`, `sudo`, the performance mistakes): section 3.15.

3.15 The ORM in module code

Reference: not presented in the session. Read it after step 5.

Section 3.14 showed the five operations from the shell. Here they are as you write them inside a module. `self.env` is available in every model method; `Asset = self.env["dswd.asset"]` gives you the model.

Domains

A domain is a list of conditions. Conditions next to each other are joined with AND; `"|"` and `"!"` are written **before** the conditions they apply to.

```
Asset = self.env["dswd.asset"]
Asset.search([("asset_type", "=", "laptop"), ("is_in_use", "=", True)])          # AND
Asset.search([("model_name", "ilike", "dell")])                                   # case-insensitive "contains"
Asset.search([("custodian_id.login", "in", ["admin", "it_lead"])])                # a dotted path follows the link
Asset.search(["|", ("asset_type", "=", "printer"), ("warranty_end_date", "!=", False)])   # OR of the next two
Asset.search(["!", ("asset_type", "=", "laptop")])                                # NOT
```

Operators you will use: `=`, `!=`, `<`, `<=`, `>`, `>=`, `in`, `not in`, `like`, `ilike`, `=like`, `child_of` (hierarchies such as areas), `any` and `not any` (conditions on the records of a One2many or Many2many). Comparing to `False` means "empty".

```
households = self.env["res.partner"].search([
    ("is_registrant", "=", True),
    ("is_group", "=", True),
    ("dswd_well_being_level_id.code", "=", "survival"),
    ("group_membership_ids", "any", [("is_ended", "=", False)]),      # has at least one current member
])
```

In Odoo 19 a domain can also be built as an object, which is easier to combine in code than nested lists:

```
from odoo.fields import Domain

in_use = Domain([("is_in_use", "=", True)])
printers_or_desktops = Domain([("asset_type", "=", "printer")]) | Domain([("asset_type", "=", "desktop")])
Asset.search(in_use & printers_or_desktops)
```

`odoo.osv.expression`, used for this in older code, is deprecated since Odoo 19; use `Domain`.

Search options, counting and existence

```
Asset.search(domain, order="warranty_end_date asc, name", limit=20, offset=40)   # page 3 of 20
Asset.search(domain, limit=1)          # still a recordset: empty if nothing matches
Asset.search_count(domain)             # SELECT count(*); there is no search(count=True) any more
bool(Asset.search(domain, limit=1))    # the cheapest "is there any?"
```

browse, exists, ensure_one, env.ref

```
asset = Asset.browse(asset_id)                  # no query until a field is read
if not asset.exists():                          # browse() of a deleted id does not fail by itself
    raise UserError(_("This asset no longer exists."))

def action_link_household(self, partner):
    self.ensure_one()                           # ValueError when called on zero or several records
    ...

managers = self.env.ref("dswd_asset.group_asset_manager")
survival = self.env.ref("dswd_wellbeing.code_well_being_survival", raise_if_not_found=False)   # None if missing
```

Buttons on a form call the method on one record; the same method called from a list on several records fails at `ensure_one()`. That is intended: it tells you the method was not written for batches.

create, write, unlink

```
assets = Asset.create([                                          # a list creates in one batch
    {"name": "LT-0101", "asset_type": "laptop"},
    {"name": "PR-0007", "asset_type": "printer", "custodian_id": officer.id},   # a Many2one takes an id
])

expired = Asset.search([("warranty_end_date", "<", today), ("is_in_use", "=", True)])
expired.write({"is_in_use": False})               # one UPDATE for the whole set
asset.custodian_id = self.env.user                # assigning a field is a write

old = Asset.search([("is_in_use", "=", False), ("purchase_date", "<", "2015-01-01")])
old.unlink()
```

Writing One2many and Many2many fields: `Command`

```
from odoo import Command, fields

household.write({"group_membership_ids": [
    Command.create({"individual": child.id}),                                  # new member line
    Command.update(membership.id, {"ended_date": fields.Datetime.now()}),      # end it; do not delete history
]})

asset.write({"tag_ids": [Command.set(codes.ids)]})     # replace all links with these
asset.write({"tag_ids": [Command.link(code.id)]})      # add one link
asset.write({"tag_ids": [Command.unlink(code.id)]})    # remove one link, keep the record
asset.write({"tag_ids": [Command.clear()]})            # remove all links
# Command.delete(id) deletes the linked record itself
```

The same commands work in XML: `eval="[Command.link(ref('base.group_user'))]"`. The old tuple form `(4, id)`, `(6, 0, ids)` still runs but this repository's hooks and rules require `Command`.

Working with recordsets

```
in_use = assets.filtered("is_in_use")                                  # by a field's truth
soon = assets.filtered(lambda a: a.warranty_end_date and a.warranty_end_date <= limit_date)
assets.mapped("name")                                                  # list of values
assets.mapped("custodian_id")                                          # recordset, duplicates removed
assets.mapped("custodian_id.email")                                    # follows the link
assets.sorted("purchase_date", reverse=True)
by_type = assets.grouped("asset_type")                                 # {"laptop": <records>, ...}
laptops | printers        # union        laptops & mine     # intersection        everything - laptops   # difference
```

Counting and summing in the database: `_read_group`

```
rows = Asset._read_group(
    [("is_in_use", "=", True)],
    groupby=["custodian_id", "asset_type"],
    aggregates=["__count"],
)
for custodian, asset_type, count in rows:            # tuples, not dictionaries; custodian is a record
    _logger.info("User %s holds %s assets of type %s", custodian.id, count, asset_type)
```

Group by dates with `"purchase_date:month"`, aggregate with `"purchase_cost:sum"`. The public `read_group` is deprecated in Odoo 19; use `_read_group` in module code. `search_read(domain, fields)` returns dictionaries and is for exports and external calls, not for logic.

Acting as someone else, and context

```
Asset.with_user(officer).search([])                          # what the officer can see (rules apply)
seq = self.env["ir.sequence"].sudo().next_by_code("dswd.asset")   # narrow and commented: why sudo is needed
Partner.with_context(active_test=False).search([...])        # include archived records
household.with_context(tracking_disable=True).write({...})   # no chatter messages (imports, tests)
Asset.with_context(default_asset_type="printer").create({...})    # default for this create only
```

Never use `sudo()` to make an `AccessError` go away. `sudo()` switches off access rows and record rules; a "fix" with `sudo()` opens data across areas. Fix the access file or the rule (3.12).

The three performance mistakes

```
# 1. N+1: one query per household
for hh in households:
    hh.dswd_member_count = Membership.search_count([("group", "=", hh.id)])

# Instead: one query for all of them
counts = dict(Membership._read_group([("group", "in", households.ids)], ["group"], ["__count"]))
for hh in households:
    hh.dswd_member_count = counts.get(hh, 0)

# 2. Writing in a loop: N updates, N recomputations
for asset in assets:
    asset.write({"is_in_use": False})
# Instead
assets.write({"is_in_use": False})

# 3. self.env.cr.commit() in module code
```

`env.cr.commit()` belongs in the shell only. In a module it breaks the rollback that protects the database when something fails, breaks tests, and breaks job retries. Long work goes to the job worker (3.17).

3.16 Model methods: overrides, errors, logs and translations

Reference: not presented in the session. Read it after step 5.

Overriding create, write and delete

Most fixes in production land inside one of these.

```
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:                                  # before super(): change the values
        vals.setdefault("notes", _("Registered through the asset register."))
    assets = super().create(vals_list)                      # the records exist from here on
    for asset in assets:                                    # after super(): act on the records
        _logger.info("Asset %s created", asset.id)
    return assets                                           # always return what super() returned

def write(self, vals):
    result = super().write(vals)
    if "custodian_id" in vals:
        for asset in self:
            asset.message_post(body=_("Custodian changed to %s.", asset.custodian_id.name))
    return result

@api.ondelete(at_uninstall=False)
def _unlink_except_in_use(self):
    if self.filtered("is_in_use"):
        raise UserError(_("Retire the asset before deleting it."))
```

- `create` receives a **list** of dictionaries (`@api.model_create_multi`). A `create(self, vals)` taking one dictionary breaks imports and batch creation.
- Code before `super()` sees values, not records; code after it sees records. Forgetting `super()` silently disables everything upstream does in that method.
- `self` in `write` can be many records; loop.
- Refusing a deletion goes in an `@api.ondelete` method, not in an `unlink` override, so uninstalling the module still works.

Errors, logs and translations

```
import logging

from odoo import _
from odoo.exceptions import UserError, ValidationError

_logger = logging.getLogger(__name__)

def action_link_household(self, partner):
    self.ensure_one()
    if not partner.is_group:
        raise UserError(_("Select a household, not an individual."))     # the user did something not allowed
    self.household_id = partner

# ValidationError: the data is wrong; raised from @api.constrains (3.8)
_logger.info("Asset %s created", asset.id)            # ids, never names or ID numbers: no personal data in logs
_("Custodian changed to %s.", asset.custodian_id.name)   # arguments after the text, never an f-string
```

The message text is what officers will report to you. Write it so they can act on it.

3.17 Scheduled actions and background jobs

Reference: not presented in the session. Read it after step 5.

```
<record id="ir_cron_dswd_asset_retire_expired" model="ir.cron">
    <field name="name">IT Assets: retire assets with an expired warranty</field>
    <field name="model_id" ref="dswd_asset.model_dswd_asset" />
    <field name="state">code</field>
    <field name="code">model._cron_retire_expired()</field>
    <field name="interval_number">1</field>
    <field name="interval_type">days</field>
</record>
```

```
def _cron_retire_expired(self):
    expired = self.search([("warranty_end_date", "<", fields.Date.context_today(self)), ("is_in_use", "=", True)])
    for start in range(0, len(expired), 1000):
        expired[start : start + 1000].with_delay(priority=20)._retire()    # one queued job per 1,000 assets

def _retire(self):
    self.write({"is_in_use": False})
```

- `with_delay()` queues the call; it does not run it now. The job worker runs it in the background, retries on failure, and records errors under Queue Jobs, not on screen.
- The module that calls `with_delay` must depend on `job_worker`.
- In tests, `with_context(queue_job__no_delay=True)` runs the job immediately, so you can assert its effect.
- Split large work into batches; never `commit()` in a loop.

3.18 Step 6: a PDF report

A report is two records in one XML file: an `ir.actions.report` that names the model, the template and the format, and a QWeb template that is ordinary HTML with `t-` attributes. Odoo renders the template to HTML, then to PDF with wkhtmltopdf, which the image includes.

```
<record id="action_report_dswd_asset" model="ir.actions.report">
    <field name="name">Asset Sheet</field>
    <field name="model">dswd.asset</field>
    <field name="report_type">qweb-pdf</field>
    <field name="report_name">dswd_asset.report_dswd_asset</field>
    <field name="report_file">dswd_asset.report_dswd_asset</field>
    <field name="print_report_name">'Asset Sheet - %s' % (object.name)</field>
    <field name="binding_model_id" ref="model_dswd_asset" />
    <field name="binding_type">report</field>
</record>

<template id="report_dswd_asset">
    <t t-call="web.html_container">
        <t t-foreach="docs" t-as="asset">
            <t t-call="web.external_layout">
                <div class="page">
                    <h2>Asset Sheet: <span t-field="asset.name" /></h2>
                    <table class="table table-sm o_main_table">
                        <tbody>
                            <tr><th>Make and Model</th><td><span t-field="asset.model_name" /></td></tr>
                            <tr><th>Type</th><td><span t-field="asset.asset_type" /></td></tr>
                            <tr><th>Custodian</th><td><span t-field="asset.custodian_id" /></td></tr>
                            <tr><th>Purchase Date</th><td><span t-field="asset.purchase_date" /></td></tr>
                            <tr><th>Warranty End</th><td><span t-field="asset.warranty_end_date" /></td></tr>
                            <tr>
                                <th>Under Warranty</th>
                                <td><t t-if="asset.is_under_warranty">Yes</t><t t-else="">No</t></td>
                            </tr>
                        </tbody>
                    </table>
                    <p class="text-muted">
                        Printed on <span t-out="context_timestamp(datetime.datetime.now()).strftime('%Y-%m-%d')" />
                        by <span t-out="user.name" />.
                    </p>
                </div>
            </t>
        </t>
    </t>
</template>
```

The file is `dswd_asset/report/dswd_asset_report.xml`; its comments are left out here.

- `binding_model_id` puts "Asset Sheet" in the Print menu of the asset list and form; `print_report_name` names the downloaded file.

- `docs` is the recordset being printed; `t-foreach` makes one page per record; `t-field` renders a field with its widget (dates, Many2one names, selections) the way the form does; `t-out` prints any expression, escaped; `t-if` and `t-else` choose what to print. Older code writes `t-esc`, which Odoo 19 deprecates: with the dev profile, every render logs a warning asking for `t-out`.
- `web.external_layout` adds the company header and footer; `web.html_container` is the page wrapper.
- A report is tested without a PDF engine by rendering the HTML: `self.env["ir.actions.report"]._render_qweb_html("dswd_asset.action_report_dswd_asset", asset.ids)`.

3.19 The four mistakes of a new module

Mistake	Symptom	Where to look
No access row	`have no access rules` in the `install` or `update` output; the menu is hidden; `AccessError` on the action URL, even as admin	`security/ir.model.access.csv`
Action without a menu, or menu without action	clean update, nothing to click	Settings > Technical > User Interface > Menu Items
A view field that is not on the model	update fails: `Field "x" does not exist in model "dswd.asset"`	compare with Settings > Technical > Fields
`_name` versus `model_id` in the access file	`No matching record found for external id 'model_dswd_...'`	`model_` plus `_name` with dots as underscores

3.20 From Module 3 to Module 4

4Ps needs a well-being level on every household. Which file do we start with?

The household already exists: it is a `res.partner` record, and that table already has hundreds of columns from several modules. A new `dswd.household` model would mean two tables, two forms, two access files, and no link to programs. Module 4 adds **one column** to the existing table instead.

And `asset_type`: adding "Tablet" to it means editing Python, releasing, and running `update` on every server. In OpenSPP a code list is **data**, edited under Settings > Vocabularies, with a namespace other systems can match. That is the first file of Module 4.

Module 3, `dswd_asset`	Module 4, `dswd_wellbeing`
`_name = "dswd.asset"`, a new table	`_inherit = "res.partner"`, columns on an existing table
`asset_type = fields.Selection([...])`	`dswd_well_being_level_id = fields.Many2one("spp.vocabulary.code")`
`<list>` and `<form>` written from scratch	`inherit_id` plus one XPath on a named anchor
an action and a menu you created	no menu: the household form already has one
groups and four access rows	access file: header only, "no new model, no rows"
`depends = ["base", "spp_security"]`	`depends = ["spp_registry", "spp_vocabulary"]`
`TransactionCase`, `create()` your own record	`RegistryCommon` hands you a household
the field is yours: `asset_type`	a field on someone else's model: prefix `dswd_`

Same files, same loop, same `test`. Three swaps matter most, each one line: `_inherit` instead of `_name`, a vocabulary Many2one instead of a Selection, and `inherit_id` instead of a view written from scratch. The other rows follow from them.

Module 4: Developing Custom Modules and Extending Odoo Models

As in Module 3, sections marked **Reference** are not presented in the session: the three kinds of inheritance, every position of an inherited view, and upgrading a module already in production. The migration examples in 4.13 follow Odoo's standard layout and were not executed.

4.1 What you can do after this module

1. Name the role of each file in a `dswd_*` module and state that a file not listed in the manifest does not exist.
2. Given a 4Ps requirement, pick the right pattern: `_inherit` on the upstream model, a `dswd_` field, a Many2one to a vocabulary code; never a new model for an attribute, never a static Selection for a code list, never an edit under `upstream/openspp/`.
3. Explain view inheritance (`inherit_id` plus one XPath on a named anchor) and recognise the error when the anchor does not exist.
4. Given a log excerpt, say which step failed and which file to open.
5. Run the tests of a module and read the pass/fail summary.

4.2 The three rules

1. **Never edit `upstream/openspp/`.** Read it; extend it from your module. The next release overwrites it.
2. **Inherit, do not redefine.** `_inherit = "res.partner"` adds to the household; `_name` would create a new table. New fields on upstream models are prefixed `dswd_`.
3. **A file not listed in `__manifest__.py` does not exist.** Data and view files load only if they are in `data`.

4.3 The requirement

The Social Welfare and Development Indicators assessment (SWDI) places every 4Ps household at one of three levels: Survival, Subsistence, Self-Sufficient. Since Executive Order 120 (July 2026) the level decides whether a household stays in the program past seven years. We add the level to the household record in OpenSPP, as a dropdown on the household Profile page.

```
dswd_wellbeing/
├── __manifest__.py                      depends on spp_registry and spp_vocabulary
├── __init__.py                          imports models
├── data/vocabulary_well_being_level.xml   the vocabulary and its three codes
├── models/__init__.py                   imports res_partner
├── models/res_partner.py                the field on the household, and a constraint
├── views/res_partner_views.xml          where the field shows on the household form
├── security/ir.model.access.csv         header only: no new model, no rows
├── tests/test_wellbeing.py              what "working" means
└── readme/DESCRIPTION.md                what the module is for
```

The build starts with a manifest like Module 3's (3.6): `"depends": ["spp_registry", "spp_vocabulary"]` and, at first, only the access file in `data`. The later steps add `data/vocabulary_well_being_level.xml` and then `views/res_partner_views.xml` to `data`. `"application": False` keeps the module behind the Apps filter: remove the filter to find it.

4.4 Vocabularies: a code list as data

OpenSPP's `spp_vocabulary` module stores code lists in two models: `spp.vocabulary`, a named list with a globally unique namespace URI, and `spp.vocabulary.code`, one code in a list with a `code`, a `display` label, a `definition` and a computed URI `namespace#code`. OpenSPP ships gender (ISO 5218), country, currency, language, occupation

(ISCO-08), education level, ID type and more. The production 4Ps modules ship about 70 of their own, 27 in the registry module alone: client status (32 statuses in 9 categories), member status, set groups, indigenous peoples groups (394 codes), well-being level, and so on, all under `urn:dswd:4ps:...`.

Why a vocabulary and not a `Selection`:

- a code list changes without a release: Settings > Vocabularies, no code change;
- deployments differ: a region can hide codes with a deployment profile;
- every code has a URI other systems can match;
- 32 client statuses and 394 indigenous peoples groups do not belong in Python.

Static selections are for internal workflow states only.

```
<odoo noupdate="1">
    <record id="vocab_well_being_level" model="spp.vocabulary">
        <field name="name">4Ps Well-Being Level</field>
        <field name="namespace_uri">urn:dswd:4ps:well-being-level</field>
        <field name="description">Household well-being level from the SWDI assessment</field>
        <field name="is_system" eval="True" />
        <field name="domain">social_assistance</field>
    </record>

    <record id="code_well_being_survival" model="spp.vocabulary.code">
        <field name="vocabulary_id" ref="vocab_well_being_level" />
        <field name="code">survival</field>
        <field name="display">Survival</field>
        <field name="definition">SWDI level 1: the household cannot meet basic needs without assistance.</field>
        <field name="target_type">group</field>
        <field name="sequence">10</field>
    </record>
    <!-- subsistence (sequence 20) and self_sufficient (sequence 30) follow the same shape -->
</odoo>
```

- `noupdate="1"`: these records are loaded once, on install, and never overwritten by an update. A bad value in such a file is caught only the first time it loads. To see an edit to one of them:
  - in development, start from a fresh database: `./odoo-project stop -v -y`, then `start`, then `install` the module;
  - on a database that must keep its data, write a migration (4.13). The ORM refuses ordinary writes to the shipped codes of a system vocabulary, so production migrations use SQL or `with_context(install_xmlid=True)`, the context the data loader itself runs with.

  Deleting the record does not work here. The Vocabularies screen refuses to delete codes of a system vocabulary ("Cannot delete codes from system vocabulary"). Deleting only the row in Settings > Technical > External Identifiers leaves the code in place, so the next `update` creates it again and fails with "Code must be unique within vocabulary".
- `domain` is a fixed selection: `core`, `social_assistance`, `social_insurance`, `labor`, `disability`, `agriculture`, `health` or `education`. Any other value fails at install with a `ParseError`.
- `is_system`: shipped codes are locked against editing in the UI; administrators add local codes next to them.
- Namespaces for your own vocabularies follow the production pattern, `urn:dswd:4ps:<name>`.

After `./odoo-project update`, Settings > Vocabularies > Manage Vocabularies lists "4Ps Well-Being Level" with three System codes.

4.5 The field on the household

```
from odoo import _, api, fields, models
from odoo.exceptions import ValidationError

WELL_BEING_NAMESPACE = "urn:dswd:4ps:well-being-level"


class ResPartner(models.Model):
    _inherit = "res.partner"

    dswd_well_being_level_id = fields.Many2one(
        comodel_name="spp.vocabulary.code",
        string="Well-Being Level",
        domain=[("namespace_uri", "=", WELL_BEING_NAMESPACE)],
        ondelete="restrict",
        help="SWDI level from the latest assessment: Survival, Subsistence or Self-Sufficient.",
    )

    @api.constrains("dswd_well_being_level_id")
    def _check_well_being_level_namespace(self):
        for partner in self.filtered("dswd_well_being_level_id"):
            if partner.dswd_well_being_level_id.namespace_uri != WELL_BEING_NAMESPACE:
                raise ValidationError(_("Well-Being Level must be a code from the 4Ps Well-Being Level vocabulary."))
```

- `_inherit = "res.partner"` with no `_name`: this class adds to the existing model. Households are `res.partner` records with `is_registrant` and `is_group` set.
- The field name starts with `dswd_`, so it never collides with a field OpenSPP adds later.
- The `domain` keeps the dropdown to the three codes. **A domain only filters the dropdown**: a write from code, an import or the API can store any code. The constraint is the real guard.
- `ondelete="restrict"`: a code in use cannot be deleted.
- This is the finished file. The session builds the field first; the constraint comes last, written test first (4.10).

After `./odoo-project update` nothing changes on screen; the field exists in Settings > Technical > Fields (search `dswd_`). A field without a view is invisible, not broken.

4.6 Three kinds of inheritance

Reference: not presented in the session. Read it after section 4.5.

```
# 1. Extend in place (Module 4): add fields and behaviour to an existing model
class ResPartner(models.Model):
    _inherit = "res.partner"

# 2. A new model built from mixins
class DswdAsset(models.Model):
    _name = "dswd.asset"
    _inherit = ["mail.thread", "mail.activity.mixin"]

# 3. Delegation: a new table that also exposes every field of a linked record
class Example(models.Model):
    _name = "dswd.example"
    _inherits = {"res.partner": "partner_id"}
    partner_id = fields.Many2one("res.partner", required=True, ondelete="cascade")
```

When you open a model and do not find the logic, look at what it inherits: the method is often in a mixin or a base class in another module. `_inherits` is rare in `dswd_*` code but appears upstream; recognise it.

4.7 The view: one XPath on a named anchor

```
<odoo>
    <record id="view_res_partner_form_wellbeing" model="ir.ui.view">
        <field name="name">res.partner.form.dswd.wellbeing</field>
        <field name="model">res.partner</field>
        <field name="inherit_id" ref="spp_registry.view_individuals_form" />
        <field name="arch" type="xml">
            <xpath expr="//group[@name='group_tags_section']" position="after">
                <group name="dswd_4ps_section" string="4Ps">
                    <field name="dswd_well_being_level_id" readonly="disabled" options="{'no_create': True, 'no_open': True}" />
                </group>
            </xpath>
        </field>
    </record>
</odoo>
```

The file in your repository starts with a comment, left out here.

- `inherit_id` names the upstream view. In `spp_registry` one form serves individuals and households; the household pages and groups are prefixed `group_` (`group_profile`, `group_tags_section`, `group_identity`).
- Find the anchor before writing the XPath: `grep -n 'name="group_' upstream/openspp/spp_registry/views/individual_views.xml`; in PowerShell, `Select-String -Path upstream\openspp\spp_registry\views\individual_views.xml -Pattern 'name="group_'`; or search the file in VS Code. Target named elements only; positional XPath breaks on the next release.
- `no_create` and `no_open` keep users from creating or editing codes from the dropdown; codes are managed under Settings > Vocabularies.
- `readonly="disabled"` follows the upstream form, which locks its own fields (names, tags) the same way: the level cannot be edited once the registrant is disabled (`disabled` is the upstream "Date Disabled" field).
- The other positions (`before`, `inside`, `replace`, `attributes`), conditional `invisible` and `readonly`, and `groups` on fields and buttons: section 4.8.

After `./odoo-project update`: Registry > Browse All (Audit) > All Groups > a household > Profile shows a "4Ps" section with the dropdown.

4.8 View inheritance: every position

Reference: not presented in the session. Read it after section 4.7.

```
<field name="model_name" position="before">
    <field name="serial_number" />
</field>

<xpath expr="//group[@name='group_lifecycle']" position="inside">
    <field name="purchase_cost" readonly="not is_in_use" />
    <field name="warranty_months" invisible="not purchase_date" />
</xpath>

<xpath expr="//field[@name='is_in_use']" position="attributes">
    <attribute name="readonly">custodian_id == False</attribute>
</xpath>

<xpath expr="//div[hasclass('oe_title')]" position="after">
    <div class="text-muted" invisible="is_in_use">This asset is retired.</div>
</xpath>

<xpath expr="//button[@name='action_assign_to_me']" position="after">
    <button name="action_open_movements" type="object" string="Movements" groups="dswd_asset.group_asset_manager" />
</xpath>

<xpath expr="//sheet" position="after">
    <chatter />
</xpath>
```

- `position` is `before`, `after`, `inside`, `replace` or `attributes`. Avoid `replace`: it breaks every other module that targets the element you removed.

- `invisible`, `readonly` and `required` take Python-like expressions on the record's fields (`not is_in_use`, `custodian_id == False`, `is_group`). The old `attrs="{...}"` syntax is gone since Odoo 17.
- `groups="..."` on a field or button hides it from everyone outside the group. It hides; it does not protect. The access file and record rules protect.
- `<chatter />` shows messages and tracked changes; the model must inherit `mail.thread`.

4.9 The security file

The module adds no model, so `security/ir.model.access.csv` holds only its header line. The repository's hooks require the file to exist in every module; an empty one says "no new model, no rows". Access to the field follows the upstream rules for `res.partner`.

4.10 The tests

Abridged; the full file is `dswd_wellbeing/tests/test_wellbeing.py`.

```
from odoo.exceptions import ValidationError
from odoo.tests import tagged

from odoo.addons.spp_registry.tests.common import RegistryCommon

NAMESPACE = "urn:dswd:4ps:well-being-level"


@tagged("post_install", "-at_install")
class TestWellBeingVocabulary(RegistryCommon):
    def test_three_levels_in_swdi_order(self):
        codes = [self.VocabCode.get_code(NAMESPACE, code) for code in ("survival", "subsistence", "self_sufficient")]
        self.assertTrue(all(codes))
        self.assertEqual([c.display for c in codes], ["Survival", "Subsistence", "Self-Sufficient"])


@tagged("post_install", "-at_install")
class TestHouseholdWellBeing(RegistryCommon):
    def test_household_stores_a_level(self):
        subsistence = self.VocabCode.get_code(NAMESPACE, "subsistence")
        self.group.dswd_well_being_level_id = subsistence
        self.assertEqual(self.group.dswd_well_being_level_id.display, "Subsistence")

    def test_level_shows_on_the_household_form(self):
        form = self.env.ref("spp_registry.view_individuals_form")
        arch = self.Partner.get_view(view_id=form.id, view_type="form")["arch"]
        self.assertIn('name="dswd_4ps_section"', arch)

    def test_rejects_a_code_from_another_vocabulary(self):
        female = self.VocabCode.get_code("urn:iso:std:iso:5218", "2")
        with self.assertRaises(ValidationError):
            self.group.dswd_well_being_level_id = female
```

- `RegistryCommon`, from `upstream/openspp/spp_registry/tests/common.py`, creates a household (`self.group`), two individuals and gives you the vocabulary model (`self.VocabCode`). Use upstream helpers before writing your own.
- `get_code(namespace, code)` and `resolve_by_uri(uri)` are the two cached lookups for codes in Python.

```
$ ./odoo-project test dswd_wellbeing
...
========== TEST RESULTS ==========
0 failed, 0 error(s) of 5 tests
Passed: 5 | Failed: 0 | Errors: 0

Full log: .../odoo-test-logs/dswd_wellbeing_unittest_<date>_<time>.log
```

The `Full log:` line names the whole log; open it when the summary is not `0 failed`.

Green tests on your laptop are where a change starts its way to `main`: a branch, a commit, a pull request, CI on GitHub, a review, a merge. Section 7.4 gives the whole path.

4.11 The five mistakes of an extension

Mistake	Symptom	One-line diagnostic
Data file written but not in manifest `data`	update clean, UI unchanged	`grep vocabulary_ dswd_wellbeing/__manifest__.py` (PowerShell: `Select-String -Path dswd_wellbeing\__manifest__.py -Pattern vocabulary_`)
Invalid value in a data file (`domain`)	`ParseError` in the `update` output, the registry fails to load	the field's definition upstream: the `domain` selection in `upstream/openspp/spp_vocabulary/models/vocabulary.py` (4.4)
Import missing in `models/__init__.py`	no error, field absent; or `Field "..." does not exist in model` once a view refers to it	Settings > Technical > Fields, search `dswd_`
Wrong XPath anchor (`group_tag_section`, `@class=`)	`Element '<xpath ...>' cannot be located in parent view`	search `name="group_` in `upstream/openspp/spp_registry/views/individual_views.xml` (4.7)
`restart` instead of `update`, or edited `noupdate` data	change never shows	new field, view or data: `update`; changed `noupdate` record: a fresh database (`stop -v -y`), or a migration (4.4)

A module that does not appear in Apps at all (folder typo, no manifest, list not refreshed): remove the "Apps" filter, turn developer mode on, click Update Apps List; the `install` output says `invalid module names, ignored`.

Three real log excerpts, as the Module 4 drill uses them:

```
ValueError: Wrong value for spp.vocabulary.domain: 'administrative'
odoo.tools.convert.ParseError: while parsing /mnt/extra-addons/custom/dswd_wellbeing/data/vocabulary_well_being_level.xml:10
```

```
odoo.tools.convert.ParseError: while parsing /mnt/extra-addons/custom/dswd_wellbeing/views/res_partner_views.xml:8
Element '<xpath expr="//group[@name=&#39;group_tag_section&#39;]">' cannot be located in parent view
```

`'` is a `'`: Odoo escapes the quotes inside the XPath when it writes the message.

```
INFO odoo.modules.loading: loading dswd_wellbeing/security/ir.model.access.csv
INFO odoo.modules.loading: loading dswd_wellbeing/views/res_partner_views.xml
ERROR ...: FAIL: TestWellBeingVocabulary.test_three_levels_in_swdi_order
AssertionError: False is not true : every level must resolve by namespace and code
```

Read a traceback from the bottom up: the first line that names a file in **your** module is where you go. Lint passes on all three; lint checks shape, not meaning.

4.12 Where this lives in production

The production 4Ps registry module ships 27 vocabularies the same way and extends `res.partner` with Many2one fields to `spp.vocabulary.code` exactly like this one: client status, member status, set group, poverty status, indigenous peoples group. Its household form is extended by inheriting the same `spp_registry` view. The well-being level vocabulary exists there too, but the household's level is copied from the latest SWDI assessment (5.8). What you built in Module 4 is the shape of every `dswd_4ps_*` module. Module 5 shows them.

`training/4ps-program-reference.md`, section 8, maps every 4Ps concept (household, grantee, client status, sets, periods, compliance, payroll, grievances) to the OpenSPP model and the production module that implements it.

4.13 Upgrading a module that is already in production

Reference: not presented in the session. Read it after the module is in production.

A module running in production holds data. Changing a stored field's name or type, or fixing `noupdate` data, needs a migration script that runs on `update`:

```
dswd_asset/
├── __manifest__.py                    "version": "19.0.1.1.0"   (bumped from 19.0.1.0.0)
└── migrations/
    └── 19.0.1.1.0/
        ├── pre-migrate.py             runs before the module's new code is loaded
        └── post-migrate.py            runs after
```

```
# migrations/19.0.1.1.0/pre-migrate.py: rename a column before the ORM sees the new field name
def migrate(cr, version):
    cr.execute("ALTER TABLE dswd_asset RENAME COLUMN model_name TO make_and_model")
```

```
# migrations/19.0.1.1.0/post-migrate.py: fill a new field, condition, on the existing rows
from odoo import SUPERUSER_ID, api


def migrate(cr, version):
    env = api.Environment(cr, SUPERUSER_ID, {})
    env["dswd.asset"].search([("condition", "=", False)]).write({"condition": "good"})
```

- The folder name must equal the new manifest `version`. A script without a version bump never runs; a version bump without the script leaves old data in the old shape.
- `version` in the `migrate` function is the version being upgraded **from** (empty on a first install).
- Raw SQL bypasses the ORM: access rules, computed-field triggers and caches. Keep it to migrations.

4.14 Exercises (Module 6, Day 3)

On Day 3 you build three modules of your own, each from a Day 2 pattern: a new module like `dswd_asset`, an extension of the household like `dswd_wellbeing`, and a customization of OpenSPP through data. The sheets are in `training/module6/`; the Module 6 chapter of this guide describes the session. Before Day 3: `./odoo-project start` and `./odoo-project test dswd_wellbeing` at 5 passed.

Module 5: DSWD-specific implementation of OpenSPP

The session discusses the production DSWD 4Ps modules, the repository `dswd-4ps-dp-openspp-v2-module`. It reads the modules through the parts of a module you met in Modules 3 and 4, in the order of the 4Ps business process, and asks one question of each: if a ticket lands here, which file do I open?

The companion to this chapter is the module catalogue, `training/module5/module-cards.md`: one card per module or module family, and an index of all 66 modules. This chapter gives the map and the points to remember; the cards give the detail. Every count, path and line number in both was checked at production commit `ad98dac9` (2026-11-06). Production moves on: when a number here disagrees with the code you have open, the code is right.

Section 5.10 is **Reference**: the habits of production code that a newcomer misreads. It is not presented in the session; read it before your first change to a `dswd_4ps_*` module.

5.1 What you can do after this module

1. Place any `dswd_4ps_*` module on the map: what part of the 4Ps process it serves and which instance it installs on.
2. Given a 4Ps event (a grantee changes, a school attendance form arrives, a payroll run, an SWDI assessment), name the module that handles it and the `spp_*` module it extends.
3. Given a ticket, name the module and the part to open first.
4. Recognise the production traps (context keys that skip checks, `x_4ps_` fields, raw SQL, `noupdate` data and install hooks, system parameters, approval state) and say why "simplifying" them is dangerous.
5. Find any module in the catalogue and read its card.
6. Use Claude Code to read and change code in your repository, and review what it wrote before you trust it (5.13).

5.2 How to read a production module

After the map (5.3), the session gives a short primer on five things the production modules use on almost every card and that Modules 3 and 4 left to the Reference sections: record rules (3.12), jobs and scheduled actions (3.17), migrations and `noupdate` data (4.13, 4.4), hooks such as `post_init_hook` and `_register_hook` (5.10), and wizards (3.10). The glossary (7.6) has a line on each.

Each card in the catalogue has the same parts:

Part	Says
The table	The 4Ps process, the instance, what it depends on, how many modules depend on it, its size
Key models	**new table** is a model with its own `_name`; **extends** is an `_inherit` on an existing model (3.7, 4.6)
The strip	Twelve boxes, one per part of a module, shaded where the module uses that part in a way worth knowing
Important points	What changes how you must work on the module
A ticket usually lands in	Files, each with a symptom in user words
Code moment, code excerpt	The file opened in the session, and the lines shown

The strip is this guide in twelve boxes:

Manifest	Models	Fields	Views	Wizards	Security	Tests	ORM	Jobs	Reports	Vocabularies	Migrations
3.6	3.7, 4.6	3.8	3.9, 4.7-4.8	3.10	3.11-3.12, 4.9	3.13, 4.10	3.15	3.17	3.18	4.4	4.13

Read a module as "which boxes are shaded": that is where its tickets land. In the session, the same parts are eight coloured cards you hold up when a symptom is read: **M** model, **V** view, **D** data or vocabulary, **S** security, **W** wizard, **J** job, **A** API, **T** test. The rule: hold up the card for the file you would open first, not the file the fix ends in.

5.3 The map

The production repository has 66 modules with a manifest. 26 cards cover 38 of them; the rest are registry tools, operations modules, and 17 sample-data, demo and test-only modules that you can set aside on a ticket.

Question	Modules	Cards
Where	`dswd_4ps_area`	2
Who	`dswd_4ps_registry`	3
Who may	`dswd_4ps_roles`, `dswd_4ps_approval_multi_group`, `dswd_4ps_approval_flow_data`	4 to 6
When	`dswd_4ps_period`	7
Change	`dswd_4ps_change_request`, `dswd_4ps_supporting_docs_data`, `dswd_4ps_field_limit`	8 to 10
Verify	`dswd_4ps_compliance`, `dswd_4ps_fds_operations`	11, 12
Pay	`dswd_4ps_payroll`, `dswd_4ps_catchup_payroll`, `dswd_4ps_retro_payroll`	13 to 15
Assess, manage	`dswd_4ps_swdi`, `dswd_4ps_cms`, `dswd_4ps_swdi_cms`	16 to 18
Connect	the bridge, `dswd_4ps_api` and its route modules, the portal APIs, `integration`, `newgrs`, `ods`, `auth`	19 to 26

Under all of them sits OpenSPP: `spp_registry`, `spp_area`, `spp_vocabulary`, `spp_security`, `spp_programs`, `spp_change_request_v2`, `spp_event_data`, `spp_case_base`, `spp_api_v2`. You have their source in `upstream/openspp/`.

Three ways to install (card 1). A deployment is chosen by which bundle module is installed:

Bundle	Installs
`dswd_4ps_starter`	Everything on one database, no bridge
`dswd_4ps_instance1`	Registry and operations: change requests, compliance, FDS, payroll, the APIs, `bridge_server`
`dswd_4ps_instance2`	Casework: SWDI, CMS, the casework portal API, `bridge_client`

Area, registry, roles, the API and auth install on both instances (with the shared bridge schemas, the portal API base, the theme and attachment hardening); period and the approval modules only with starter or instance 1. Manifest `excludes` enforce the split: 26 modules exclude `dswd_4ps_bridge_client`, 8 exclude `dswd_4ps_bridge_server`. A manifest is the map of a deployment (3.6).

5.4 Foundation: where, who, who may, when

`dswd_4ps_area` (card 2) loads the PSGC geography, about 42,000 barangays, into `spp.area`, and limits what each user sees to their center areas.

- The `post_init_hook` loads the barangays as job-worker batches at install, and runs only then: every later data fix is a migration.
- Area search is fail-closed: a user with no center areas sees no areas.
- `area_scope_ids` replaced a `child_of` rule that took 15 to 20 seconds per dashboard tile at 7.4 million households; it takes 99 ms. Do not simplify it back.

`dswd_4ps_registry` (card 3) is the household and member record: `listahanan_id`, `client_status_id`, `grantee_id`, `member_id_4ps` and many more on `res.partner`.

- 27 vocabularies, among them 32 client statuses in 9 categories: a status's `parent_id` is its category (4.4 at scale).
- Writes go through `_4ps_write_hook`, which `_register_hook` wires into the audit decorator; a breakpoint in `write()` may never be reached.
- Status 3 is a permanent exit: a normal user's write cannot move a household out of it.
- Three field-name styles: no prefix, `x_4ps_` and `dswd_export_`. Search all three before adding a field.

`dswd_4ps_roles` (card 4) turns the 4Ps organisation into groups: 54 groups and 48 roles. The POO, RPMO and NPMO umbrella groups are the approval tiers; sub-roles such as MSWDO and SWO III imply them through `implied_ids`, the pattern of 3.11. Local roles take their areas from the identity provider. Every group, role and rule file is `noupdate`, so every change ships a migration: 20 so far.

`dswd_4ps_approval_multi_group` and `dswd_4ps_approval_flow_data` (cards 5 and 6). The first lets a tier name several groups, any of which may approve; it replaces two upstream constraints without calling `super()`, so a check upstream adds later does not run here. The second is data only: 21 approval definitions, 19 linked to 6 change-request types and chosen by CEL conditions on the status transition. A transition no definition matches approves on submit. Reinstate Household keeps a catch-all definition, so every reinstatement needs approval; Change Address has no linked definition, so it approves on submit.

`dswd_4ps_period` (card 7): one compliance period per month, and the Philippine working-day calendar. It fails loud: a year not marked Seeded raises an error rather than give a deadline that is too early. Each December an administrator enters next year's holidays and marks the year Seeded.

5.5 Change requests: the Beneficiary Update System

`dswd_4ps_change_request` (card 8, the largest module) adds the 4Ps types to OpenSPP's change requests, with `dswd_4ps_supporting_docs_data` (card 9) and `dswd_4ps_field_limit` (card 10). Learn the upstream shape first; 4Ps does not change it:

Part	Is
Type	A data record, `spp.change.request.type`, in a `data/cr_type_*.xml` file
Detail model	The proposed values, one model per type (`models/cr_detail_*.py`)
Apply strategy	Validates, previews and applies the change
Approval	`spp.approval.mixin`, multi-tier, routed by the approval definitions

- 20 new types. 9 are pure field mappings; 11 have a Python apply model.
- Requests are batched under a ticket number, and a `cancelled` state is added to the approval mixin.
- Some requests create others: a newborn's Add Member creates Update Pregnancy and Update Member Additional Information requests for the parents.
- Record access is checked in code (`models/cr_parent_access.py`), not by a record rule. Its docstring says why.
- The module reads 14 context keys of its own; 11 skip or override a check (5.10).
- `dswd_4ps_supporting_docs_data` says which documents each type needs; Add Member needs any one of seven. `dswd_4ps_field_limit` caps how often a field may change, in a `write` override.

Adding a field to a request type touches more than one file: the detail model and its view; the field mapping in the type's data file, which is `noupdate`, so a migration and a version bump; the API schema and service if the mobile app sends the field; the supporting-documents trigger path; and the tests.

5.6 Compliance and Family Development Sessions

`dswd_4ps_compliance` (card 11), the Compliance Verification System, turns monthly school attendance, health visits and deworming into compliance status rows that payroll reads.

- Forms arrive through the `/cv` API (the router is in `dswd_4ps_api_compliance`) as `spp.event.data` events.
- The event's `create` hands the work to a job with `with_delay` (3.17); the job takes a per-household lock, writes one row and marks the household; a daily reconcile finishes the household. Status rows are a cache rebuilt from events.
- When a submitted form changes nothing, look at the event and its job before the code.
- The school threshold is the constant 85 in `models/compliance_pipeline.py`. A system parameter with a similar name only feeds display fields.

`dswd_4ps_fds_operations` (card 12): the session library, parent groups (up to `pg_max_members`, 30 by default), plans with approval, attendance and the FDS Accomplishment Report. FDS attendance arrives as session attendance, not as event data. An equivalency creates a synthetic session, so the compliance pipeline needs no special case.

5.7 Payroll: what not to break

`dswd_4ps_payroll` (card 13) is about 50,000 lines and the riskiest code in the repository; read before you touch.

- The cycle run is a state machine (`STATE_SELECTION` in `models/payroll_cycle_run.py`). Once credit instructions exist, cancel and rewind are refused: the point of no return.
- The snapshot, which the code calls NAPA, is sealed: `models/snapshot_beneficiary.py` refuses writes, and every later step reads it. A name corrected after the seal does not appear on that cycle's payroll list, by design.
- Grant rates are never deleted. Five job channels; about 110 raw SQL calls.

`dswd_4ps_catchup_payroll` (card 14) pays households a regular cycle left out, with the same state-machine pattern and its own point of no return. **`dswd_4ps_retro_payroll`** (card 15) keeps overpayment and underpayment ledgers and reuses the regular run.

5.8 Casework and the bridge

- **`dswd_4ps_swdi`** (card 16): the yearly well-being assessment. The level comes from the total score (level 3 from 2.83, level 2 from 1.83). This is the well-being level of Module 4, computed: in production the household's level is copied from the latest assessment, not chosen from a dropdown.
- **`dswd_4ps_cms`** (card 17): Kilos-Unlad case management on `spp_case_base`, with 117 record rules for six groups, each scoped by the user's own center areas (3.12 at production scale).
- **`dswd_4ps_swdi_cms`** (card 18): seven triggers, each a data record, turn assessment results into cases. Approving an assessment queues an event; a job creates the case and retries after 5 minutes, 15 minutes and 1 hour. A failed event stays as a row in state `failed`, with a Retry action.
- **The bridge** (card 19): the registry instance is the master. Casework looks households up and imports them synchronously, waiting at most 5 seconds; case events flow back through jobs.

5.9 Integration

`dswd_4ps_api` (card 20) adds the 4Ps endpoints to OpenSPP's API application. A router runs as the caller (`authenticated_env`), checks a scope, finds records by identifier, never by database id, and hands the work to a service. Record rules apply to API traffic while the system parameter `dswd_4ps_api.enforce_rbac` is on, as it is seeded.

Card	Module	In one line
21	`api_change_request`, `api_compliance`, `api_fds`, `api_field_limit`, `retro_api`, `swdi_api`	Each adds its routers with a short `_inherit` and `super()`
22	`portal_api_base`, `portal_api`, `portal_api_casework`, `payroll_api`	What the beneficiary portal reads
23	`dswd_4ps_integration`	An outbound event log the portal polls
24	`dswd_4ps_newgrs`	Grievance status in, profile updates out, three flags
25	`dswd_4ps_ods`	A read-only client for the old Pantawid ODS
26	`dswd_4ps_auth`	Keycloak sign-in; a user gets their areas from the claims

5.10 Reading production code

Reference: not presented in the session. Read it before your first change to a production module.

The production modules use everything in Modules 3 and 4, plus a few things a newcomer misreads. Recognise them before changing them.

- **Context keys that skip checks.** Calls like `with_context(skip_some_check=True)` turn off checks inside overrides; the change-request module alone reads 14 of its own. Removing one, or calling a method without it, can cause duplicate records or endless recursion. Search for where the key is read before touching it, and never pass one to make a test pass.
- **The logic is not where you opened it.** Detail models inherit base classes and mixins; a write may go through a hook registered by `_register_hook` rather than through `write()`. Follow `_inherit`, and search for the method name across the repository.
- **Field and model names.** Odoo reserves the `x_` prefix for fields created in the user interface; existing production fields use `x_4ps_`, others have no prefix or `dswd_export_`, and some new models carry `spp.` names. New fields use `dswd_`.
- **Raw SQL in models.** Payroll and compliance use `cr.execute` for speed, with `flush_model()` before and `invalidate_model()` after so the ORM sees the result. Do not "simplify" them into ORM loops over millions of rows, and do not add raw SQL elsewhere.
- **Data and hooks that run once.** `noupdate="1"` data and `post_init_hook` run at install only. Changing them changes nothing on an existing database; ship a migration (4.13).
- **Configuration in settings.** Thresholds and switches are read from `ir.config_parameter` (`self.env["ir.config_parameter"].sudo().get_param("dswd_4ps.some_key")`), so "why does it behave like this" may be answered in Settings > Technical > System Parameters. Check that the code reads the parameter where you think: a parameter may feed only a display field while the rule uses a constant.
- **Approval state.** Records with `spp.approval.mixin` keep their state consistent with review records; never set `approval_state` directly.
- **Help texts, docstrings and manifests drift.** A manifest may say six triggers where the data has seven. Trust the data and the code.
- **Tests are the specification.** Most production modules have hundreds of tests. Before changing behaviour, find the test that describes it.

5.11 How to approach a 4Ps ticket

1. Read the symptom in the user's words, and reproduce it.
2. Find the module on the map (5.3) and open its card.
3. Read the strip: the shaded boxes are where its tickets land. Pick the first file.

4. Before changing code, check what runs around it: `noupdate` data, context keys, jobs and their state, system parameters.
5. Write a failing test that reproduces the ticket, then fix, then run the module's tests.

A worked example. "The list of reasons for a grantee change needs 'working abroad'." A list of reasons is a code list: **D**, a vocabulary (4.4). The grantee belongs to the registry (card 3), and its data files include `dswd_4ps_registry/data/vocabulary_grantee_change_reasons.xml`, which has `noupdate="1"`. Adding the code to the file is enough for a fresh database; an existing database also needs a migration that creates the code, and a version bump (4.13). Then a test that the code exists. One line of XML; three files for a release.

In the session you practise this in pairs on eight training tickets (the triage sheet): for each, the module, the category card and the first file. The answers are discussed at the end of the triage.

5.12 Not today

The modules on no card are in the catalogue index, for the post-training sessions: registry tools (`registry_import`, `export`, `ecr`, `hazard`, `account_creation`), operations (`theme`, `attachment_hardening`, `post_sql_repair`, `pg_partition`), dashboards (`swdi_dashboard`, `cms_nas`) and 17 sample-data, demo and test-only modules.

5.13 Claude Code in development

Claude Code is an AI assistant that works in your terminal, inside your repository: it reads files, runs commands such as `./odoo-project test`, and writes changes, asking before each action. It is good at reading a large code base, at the repetitive parts of a change, and at running the loop. Deciding what to build, reviewing what it wrote, testing and owning the change stay with you.

What it reads before it answers. Your repository is set up for it:

File	What it gives Claude Code
`CLAUDE.md`	The project's rules: naming, extension patterns, the checklist, the commands, the known pitfalls
`.claude/rules/*.md`	Loaded when it edits matching files: Python models, XML views and data, security files, tests, manifests
`.claude/agents/*.md`	Specialists it can call: `odoo-developer`, `code-reviewer`, `ux-expert`, `code-simplifier`, `verify-module`
`.claude/commands/*.md`	Commands you type: `/implement` (carry out a plan with its agents), `/verify-tests`, `/analyze`, `/expert-review`, `/commit`, `/pr`
`.claude/settings.json`	Allowed commands; no edits under `upstream/` (denied); after every edit a lint hook runs ruff, Prettier and the Odoo checks on the file; no AI attribution in commits

`docs/guides/claude-code-for-developers.md` describes each of them and your personal settings.

Ask it to read, then check. "Where does OpenSPP keep a household's members, and how do I get each member's age?" is a good first question in a code base you do not know. Then open the files it names: the answer is only as good as what you verified. Two traps from the session:

- Upstream's `age` on an individual is a **text** field (`"5"`, or `"No Birthdate!"`), computed from `birthdate`. Code that counts or compares by `age` is wrong; use `birthdate`.
- A name you have seen is not a field. `children_count` appears in OpenSPP only as an example in a help text and as a Studio variable name; the household has no such field.

Test first, with it. Ask for the failing test first, run it, then ask for the change, then run the module's tests yourself and read the summary line (3.13). Say "test first" in the request: `/implement` writes the tests after the code unless you ask. A rule the session met: OpenSPP refuses a membership whose `ended_date` is earlier than its `start_date`, and `start_date` defaults to now, so a test that ends a membership also sets `start_date` in the past.

Review what it wrote. Before you run or commit anything:

1. Does the test fail without the change and pass with it? Does it test the behaviour, not the implementation?
2. Is every new file in the manifest? Is every new field in a view, if users need it?
3. Is anything changed under `upstream/openspp/`? (It should be blocked; check anyway.)
4. Do names follow 7.1 (`dswd_` prefix, model and XML id patterns)?
5. Does it claim something about the code you have not seen in a file? Open the file.
6. Did it run the module's tests, and did you read the summary line yourself?
7. Nothing secret or personal in the prompt or the diff: no beneficiary data, passwords, tokens or keys.
8. Commit messages and pull requests carry no AI attribution in this project (`settings.json` turns it off; `/commit` and `/pr` repeat it).

Safe use. What you type goes to an external AI service. Never put beneficiary or other personal data, passwords, tokens, keys, database dumps, or code you may not share into a prompt or a file you ask it to read. Read every permission request before you approve it, and never approve a command you do not understand.

From change to merge. `/commit` writes a conventional commit message and runs the pre-commit hooks; `/pr` opens the pull request and fills its template. CI, review and merge are the same as for a change you wrote yourself (7.4).

Module 6: Hands-on Activity

Day 3, 8:30 to 10:30. You build three modules of your own on your own stack, one for each skill of Day 2: a new module (development), an extension of an OpenSPP model (extending), and OpenSPP configured through data (customization). Each is a 4Ps request, and each starts from a module you already have: `dswd_asset` from Module 3 or `dswd_wellbeing` from Module 4. The last 30 minutes are a discussion of the work done in the room.

The exercise sheets are in `training/module6/`. They give what to build, which file to copy, the command after each step and what you should see. They do not print the finished files: writing them is the exercise.

6.1 What you can do after this module

1. Create an Odoo module from scratch, install it on your stack and find it on screen.
2. Extend an OpenSPP model with a new field and show it on an OpenSPP view, without touching `upstream/openspp/`.
3. Customize OpenSPP through data, and prove the configuration with a test.
4. Run the loop on your own and read the output when something fails.

6.2 Before 8:30

```
.\odoo-project start
.\odoo-project test dswd_wellbeing
```

Log in and wait for `0 failed, 0 error(s) of 5 tests`. The first test run of the day is the slowest; doing it before the session keeps it out of the exercises. If your laptop cannot start the stack, sit with a pair whose laptop can: every exercise is done in pairs, one person typing and one reading the sheet and the output, swapping at each exercise.

Then create the branch for the morning's work:

```
git switch main
git pull
git switch -c module6
```

All three exercises go on this one branch, with a commit at the end of each. (Normally each change has its own branch, 7.4. Here the three modules stay on one branch because your stack runs what is on disk: switching to another branch would take an installed module's folder away from under it.)

6.3 The three exercises

#	Skill	Module	The 4Ps request	Copy from	Sheet	Time
1	Development	`dswd_fds_session`	A register of Family Development Sessions held in each barangay	`dswd_asset`	`training/module6/exercise-1-development.md`	40 min
2	Extending	`dswd_listahanan`	A Listahanan ID on every household, never shared by two households	`dswd_wellbeing`	`training/module6/exercise-2-extending.md`	30 min

#	Skill	Module	The 4Ps request	Copy from	Sheet	Time
3	Customization	`dswd_id_types`	PhilSys as an ID type in OpenSPP, with a check of the number's format	`dswd_wellbeing`	`training/module6/exercise-3-customization.md`	15 min

What each one practises, and where this guide explains it:

- **Exercise 1** is Module 3 again with a new subject: manifest (3.6), model and fields (3.7, 3.8), views, action and menu (3.9), access rows (3.11), a constraint (3.8) and tests (3.13). It uses OpenSPP's own registry groups for access instead of new ones, and an OpenSPP model, `spp.area`, for the barangay.
- **Exercise 2** is Module 4 with a plain field instead of a vocabulary: `_inherit` (4.6), the `dswd_` prefix (4.2), one XPath on a named anchor (4.7), and a database constraint, `models.Constraint` (3.8), so that two households can never share an ID even if two users save at the same moment.
- **Exercise 3** has no model and no view: one record in a vocabulary OpenSPP already has (4.4), and OpenSPP's own code does the checking. This is how most 4Ps requirements are met: configuration through data, not new code.

If the session ends before you finish exercise 3, finish it afterwards: the sheet is complete on its own.

6.4 Done means three proofs

For every exercise:

1. `.\odoo-project install <module>` prints `Install complete. Restarting Odoo...`
2. `.\odoo-project test <module>` prints `0 failed`
3. You can show the result on screen

Read every command's output to the end. If it does not print the "complete" line, the command failed, and the browser still shows the old screens: a failed `install` or `update` leaves the previous version of the server running.

Then commit the exercise (7.4, steps 8 and 9): `git add` the module's folder (`git add dswd_fds_session`), `git status`, and `git commit -m "feat(fds): register FDS sessions"` (or `feat(listahanan): ...`, `feat(id-types): ...`). Adding the folder is the one exception to 7.4's "add files by name": a new module's folder holds only that module's files, and `git status` shows them before the commit. The hooks run; if one changes a file, add it and commit again.

At 9:55, whatever exercise you are on, commit what works, then push the branch and open a pull request (7.4, steps 10 and 11):

```
git push -u origin module6
```

CI starts on GitHub. Its first run builds the image, so it finishes after the session; check it later in the pull request's **Checks** tab and, when it is green, merge it, delete the branch, then `git switch main`, `git pull`, `git branch -d module6` (steps 12 to 15). A review is not required for this exercise.

If you finish an exercise after the session, commit it and run `git push` again: the pull request updates itself and CI runs again.

6.5 When something goes wrong

Each sheet ends with a table of the errors you are most likely to meet. Three habits cover most of them:

- **Find the first line that names your module.** In a long traceback, read from the bottom up to the first line with a path in your module folder (4.11).

- **`CRITICAL` is the symptom.** When `test` prints only `CRITICAL: ... Failed to load registry` or `Couldn't load module`, open the full log it names on its last lines and search for `Error`. The line you need is there, for example `FileNotFoundError` for a file the manifest lists but that does not exist.
- **A module copied from another keeps its names.** After copying `dswd_asset` or `dswd_wellbeing`, search the new folder for the old name (VS Code: right-click the folder, Find in Folder). Leftover ids, model names and imports cause most first errors.

6.6 After the training

The three modules are yours: once CI is green, merge the pull request and bring your laptop up to date (7.4, steps 14 and 15). They are small, working examples of the three kinds of change you will make to the 4Ps system, and each one has a production counterpart you can now read: the FDS operations module (Module 5, card 12), the Listahanan ID on the registry (card 3), and the PhilSys, 4Ps ID card and PhilHealth ID types in the registry module's data (card 3).

7. Reference

7.1 Naming rules of this repository

Thing	Rule	Example
Module folder	`dswd_<domain>` or `dswd_<domain>_<feature>`	`dswd_asset`, `dswd_wellbeing`
New model	`dswd.<domain>` or `dswd.<domain>.<entity>`	`dswd.asset`
Field on an upstream model	prefix `dswd_`	`dswd_well_being_level_id`
Many2one field	ends in `_id`; One2many and Many2many in `_ids`	`custodian_id`
Boolean field	starts with `is_`, `has_` or `can_`	`is_in_use`
View XML id	`view_<model>_<type>` (`list`, `form`, `search`)	`view_dswd_asset_form`
Action XML id	`action_<model>`	`action_dswd_asset`
Menu XML id	`menu_<model>` or `menu_<model>_<purpose>`	`menu_dswd_asset_root`
Group XML id	`group_<domain>_viewer`, `group_<domain>_officer`, `group_<domain>_manager`	`group_asset_officer`
Access row id	`access_<model>_<group>`	`access_dswd_asset_officer`
Vocabulary namespace	`urn:dswd:4ps:<name>`	`urn:dswd:4ps:well-being-level`

The pre-commit hooks enforce most of these; `./odoo-project lint` runs the formatters on changed files, and `git commit` runs the full set.

7.2 Odoo 19 notes

- The list view tag is `<list>`, not `<tree>`.
- Every model needs `_description` (Odoo logs a warning without it; this repository requires it).
- `_sql_constraints` is replaced by `models.Constraint`, `models.Index` and `models.UniqueIndex` (3.8).
- `odoo.osv.expression` is deprecated: build domains with `odoo.fields.Domain` (3.15).
- The public `read_group` is deprecated: use `_read_group` (3.15).
- Groups carry a `privilege_id` so they appear on the user form; users link to groups through `group_ids`.
- Relational writes, in Python and in XML, use `Command.link`, `Command.set`, `Command.create`, not the old `(4, id)` and `(6, 0, ids)` tuples.
- `invisible`, `readonly` and `required` in views take expressions; `attrs=` is gone.
- Search methods of computed fields receive `in` / `not in` (3.8).
- Search-view `<group>` elements take no `string` or `expand` attributes.
- XPath on a class uses `hasclass('name')`, not `@class='name'`.

7.3 Things that break the whole stack

- A top-level folder named `forms`, `helpers`, `mixins`, `utilities` or `vendor` in your repository: Odoo resolves Bootstrap's stylesheet imports against the addons root, and every page loses its styling. `doctor` reports it, `start` refuses to run, the hooks block it. Rename the folder.
- Odoo keeps a compiled stylesheet bundle while its version hash matches, and the hash comes from the source files, not from a successful compile. After any broken build, `./odoo-project resetassets`.
- `./odoo-project update` cannot install a new module; `install` can. A first `start` on a fresh database installs OpenSPP and waits; do not run `install` until it has printed the URL.

7.4 From a change to a merge

Every change to a module, during the training and after it, takes the same path. Your laptop proves the change works; GitHub proves it again on a clean machine, lets someone else read it, and keeps the history.

#	Step	Command or place
1	Start from an up-to-date `main`	`git switch main`, then `git pull`
2	Create a branch for the change	`git switch -c feat/fds-session`
3	Write the test, then the code	VS Code
4	Deploy it on your own stack	`.\odoo-project install <module>` (new) or `.\odoo-project update` (changed)
5	Check it on screen	the browser
6	Run the module's tests	`.\odoo-project test <module>`: `0 failed, 0 error(s)`
7	Lint and check the security files	`.\odoo-project lint`, `.\odoo-project audit-security`
8	Stage the files, by name	`git add <files>`, then `git status`
9	Commit; the pre-commit hooks run	`git commit -m "feat(fds): register FDS sessions"`
10	Push the branch	`git push -u origin feat/fds-session`
11	Open a pull request	GitHub: the link `git push` prints, or **Compare & pull request**
12	Read CI	the pull request's **Checks** tab
13	Review	a second person reads the diff
14	Merge	**Merge pull request**, then **Delete branch**
15	Bring your laptop up to date	`git switch main`, `git pull`, `git branch -d feat/fds-session`

One change, one branch. Never commit to `main` directly. Name the branch after the change (`feat/...` for something new, `fix/...` for a correction, `docs/...` for text) and write commit messages the same way: `feat(scope): ...`, `fix(scope): ...`, `docs(scope): ...`, in the imperative ("add", not "added").

Stage before the hooks run. pre-commit checks what is staged, not what is on disk: run it, or commit, after `git add`. If a hook changes a file (Prettier reformats a table, for example), the commit stops: `git add` the file again and commit again. Add files by name rather than whole folders, and read `git status` before every commit. Without `-m`, Git opens its default editor, which on Git for Windows is Vim unless you chose another when installing (type `:q!` and Enter to leave it).

The pull request. Its base is `main`. Your repository has a pull request template: answer its headings (why the change is needed, how it was done, which tests you added and ran). One pull request per change.

CI. Every pull request to `main` runs two workflows on GitHub's machines:

- **pre-commit**: the same hooks as on your laptop, on every file.
- **CI**: builds the OpenSPP image, then installs each module the pull request changed into a fresh database and runs its tests, with a coverage check (at least 85% of the module's Python lines, `fail_under` in `pyproject.toml`). The first run builds the image from scratch, 10 to 25 minutes; later runs reuse the cache.

A pull request is ready when every check is green. Private repositories have a monthly allowance of GitHub Actions minutes; if runs stop starting, look at your account's billing settings.

When CI is red. Open the failed check, then **Details**: the log of the step that failed. Read it bottom up, as you read an Odoo log (4.11); the test job also uploads the full test log as an artefact. Reproduce it on your laptop with the same command (`.\odoo-project test <module>`, or `pre-commit run --all-files`), fix it, commit and push again: the pull request updates itself and CI runs again. Never merge red.

Review. Before a merge, a second person reads the diff: are there tests, and do they test the change? Is every new file in the manifest? Is nothing edited under `upstream/openspp/`? Do names follow 7.1? No passwords or keys? The reviewer comments on lines in the **Files changed** tab and approves when satisfied; the author answers by pushing new commits. In your own repository, add the reviewer under **Settings > Collaborators**. A project can also protect `main` (**Settings > Branches**): merges only through a pull request, only with green checks, only after an approval. When an assistant such as Claude Code wrote the change, you are its first reviewer: read the diff before you run anything (is the record in the right file and vocabulary, is every file still in the manifest, does the test assert something that could fail?), run the tests, then keep or discard it. The tool types; you decide. Section 5.13 has the full review checklist.

Merge, then sync. Merge pull request puts the change on `main` on GitHub; delete the branch there. Your laptop does not know yet: step 15 brings it up to date. If the merge changed a module installed on your stack, run `.\odoo-project update`. The next change starts again at step 1.

7.5 Where to read more

- `docs/guides/windows-setup.md`, section "Common Errors": what to do when Docker, WSL, a port or the CLI fails on Windows.
- `docs/guides/odoo-project-cli.md`: every `./odoo-project` command, and its "Troubleshooting" section.
- `docs/guides/extending-openspp.md`: the companion of Module 4, step by step.
- `docs/guides/module-development.md`: the full scaffold for a new module, starter and variant patterns.
- `docs/guides/testing-guide.md` and `docs/principles/testing.md`.
- `docs/principles/access-rights.md`: the three-tier security model.
- `upstream/openspp/<module>/readme/DESCRIPTION.md` and `upstream/openspp/AGENTS.md`: what each OpenSPP module does and the upstream conventions.
- `training/4ps-program-reference.md`: the 4Ps program in the terms the code uses.
- https://docs.openspp.org and https://www.odoo.com/documentation/19.0/developer.html.

7.6 Glossary

Term	Meaning
Access row	One line of `ir.model.access.csv`: what one group may do on one model.
Action, menu	An action names a model and its views; a menu points at an action.
Addons path	The folders Odoo scans for modules; your repository root is one of them inside the container.
Approval mixin	`spp.approval.mixin`: multi-tier approval on a record, such as a change request; its state must match its review records (5.5, 5.10).
`auto_install`	Manifest key: Odoo installs the module by itself once all its dependencies are installed (3.6).
Bridge	The link between the registry instance and the casework instance: lookups and imports one way, case events back through jobs (5.8).
CEL	Common Expression Language: the conditions that pick an approval definition in OpenSPP.
Center area	An area a 4Ps user is assigned to; the area rules show the user only what lies in their center areas (5.4).
Context key	A flag passed down with `with_context(...)`; production code reads some to skip a check (3.15, 5.10).
Cron, scheduled action	An `ir.cron` record that runs a model method on a schedule, for example daily (3.17).
Domain	A list of `(field, operator, value)` conditions passed to `search` and to filters.
`excludes`	Manifest key: modules that cannot be installed on the same database as this one (5.3).

Term	Meaning
Group, privilege, category	A permission tier; what the user form shows for it; the heading it sits under.
Hook	Code Odoo calls at a fixed moment: `post_init_hook` once, right after the module installs; `_register_hook` each time the server loads the module (5.4, 5.10).
`_inherit`	Add fields and behaviour to an existing model.
Inherited view	A view that modifies another with XPath on named elements.
Install bundle	A module with no models whose `depends` installs a deployment: `starter`, `instance1`, `instance2` (5.3).
Instance	One of the two production databases: instance 1, registry and operations; instance 2, casework (5.3).
Job, job worker	Work queued with `with_delay()` and run later by a background worker, on a named channel (3.17).
Manifest	`__manifest__.py`: the module's name, dependencies and data files.
Migration	A script in `migrations/<version>/` that runs on `update` when the manifest version goes up: how a data or schema change reaches a database that already holds data (4.13).
Model, field	A table and a column, declared in Python.
MVC	Model, view, controller: in Odoo, `models.Model`, XML views and QWeb templates, and actions, menus and model methods.
`noupdate`	Data loaded once on install and never overwritten by an update.
ORM	Odoo's object layer: `create`, `write`, `search`, `unlink` on a model instead of SQL.
PSGC	Philippine Standard Geographic Code: region, province, city or municipality, barangay. `dswd_4ps_area` loads it.
QWeb	Odoo's template engine: HTML with `t-` attributes, used by reports and by the web client.
Record rule	An `ir.rule` record: a domain saying which rows of a model a group may see or change (3.12).
Registrant, household, group	A `res.partner` with `is_registrant`; a household is also `is_group`.
Router, scope	In the API, a router is the code behind a set of endpoints; a scope is the permission the calling client must hold (5.9).
Superuser, `sudo()`	The superuser ignores access rows and record rules, and `sudo()` runs code as it. Tests and the shell run as the superuser; `admin` is not it (3.9, 3.15).
SWDI	Social Welfare and Development Indicators: the 4Ps yearly well-being assessment, levels 1 to 3.
System parameter	A key and a value in Settings > Technical > System Parameters (`ir.config_parameter`), read by code for switches and thresholds (5.10).
`TransactionCase`	The Odoo test base class: each test runs in a rolled-back transaction.
View	XML that draws a model as a list, a form or a search box; an `ir.ui.view` record.
Vocabulary, code	OpenSPP's code lists as data, with namespace URIs; `spp.vocabulary` and `spp.vocabulary.code`.
Wizard	A dialog backed by a temporary model, a `TransientModel` (3.10).
XML id	The address of a record defined in a data file, `module.name`; used by `ref=` and `env.ref`.

The 4Ps program terms (C/ML, FDS, CVS, BUS, POO, RPMO, NPMO, Kilos-Unlad and others) are in the glossary of `training/4ps-program-reference.md`, section 9.